

Radical Denesting with Strad.wl

**Second unified analysis: three reviews of the first corrected version,
the probing session, and the second corrected version StradFixed2.wl**

Analysis session for Vladimir Reshetnikov

RadicalDenest repository, src/code-review/unified-B

5 September 2026

Abstract

The first corrected version of the radical denester, StradFixed.wl, was itself reviewed three times (src/code-review/review-4, review-5, review-6), and a probing session recorded fifteen denestable inputs that it leaves unchanged (src/corrected/KNOWN_GAPS.md). This document analyses the three reviews, consolidates their findings with the probing results into a numbered catalogue of 23 issues, and presents src/corrected/StradFixed2.wl, a rewrite that addresses them. The rewrite makes one acceptance gate unavoidable for every candidate, whatever produced it; runs every call inside one time and memory region with every expensive kernel operation bounded; resolves and validates options once per entry point; replaces the marker wrapper and the custom factorizer by a congruence walk over arithmetic hosts; adds exact fast paths for direct, indirect, Gaussian, cubic-in-quadratic, Honsbeek and multi-surd square roots; adds negative-radical separation, Kummer linear factors and index reduction for composite indices; and bounds the multiplier queue at a single admission point with a patience rule.

Everything reported here was executed in Wolfram 15.0.1. The unified-A battery of 116 inputs and the ten randomized families (340 inputs) produce no wrong value; all fifteen previously missed inputs are now denested; the 144 regression tests that translate the three reviews' suites pass; and the reviews' own native suites, which none of the reviews could run, were executed against the reviews' proposed implementations with the results recorded in Section 6.6. Remaining limitations are stated in Section 7.

Evidence. All executions were performed in 15.0.1 for Microsoft Windows (64-bit) (July 2, 2026) through wolframscript, one kernel at a time. Every table is generated mechanically from the recorded runs in logs/ by harness/export_tables2.wl. Statements about the reviews cite their finding identifiers; statements about StradFixed.wl cite the first unified analysis in src/code-review/unified-A ("unified-A" below).

Contents

1	Scope, material, and method	3
1.1	The object under analysis	3
1.2	What the reviews could and could not see	3
1.3	What this document adds	4
2	The three reviews	4
2.1	Where they agree	4
2.2	Findings unique to one review	4
2.3	Claims that we weigh differently	5
2.4	Kernel reproductions of the reviews' probes	5

3	The consolidated catalogue	6
4	The design of StradFixed2.wl	7
4.1	One acceptance gate	8
4.2	Session, budgets and bounded operations	8
4.3	Options	8
4.4	Grammar, depth and cost	8
4.5	Certification	9
4.6	Traversal by congruence	9
4.7	Fast paths	9
4.8	Negative radicands, Kummer factors and index reduction	10
4.9	The multiplier queue	10
4.10	Helpers	10
4.11	Reporting	11
5	The mathematics of the new fast paths	11
5.1	Direct and indirect square roots	11
5.2	Gaussian square roots	11
5.3	Cube roots inside a real quadratic field	11
5.4	Honsbeek's square root of two cube roots	12
5.5	Multi-surd square roots	12
5.6	Negative radicands and the linear factors of $X^q - \rho$	12
6	Experiments	12
6.1	Setup	12
6.2	The unified-A battery	13
6.3	Randomized families	18
6.4	The fifteen previously missed inputs	18
6.5	The regression suite	20
6.6	The reviews' own suites against their own proposals	20
6.7	Timing	20
7	Compatibility changes and remaining limitations	21
8	Future work	21
A	Source of StradFixed2.wl	22
B	Files of this directory	33

1 Scope, material, and method

1.1 The object under analysis

`src/corrected/StradFixed.wl` (504 lines, context `RadicalDenest` ‘) is the corrected version of `src/original/Strad.wl` produced in unified-A. It keeps the architecture of the original (a marker wrapper, a multiplier search over minimal polynomials and polynomial GCDs, a roots-of-unity orbit) and adds exact certification of every candidate against the original problem, a branch-safe factorizer, budgets, a visited set and a quadratic-surd fast path. Unified-A evaluated it on a battery of about 90 hand-chosen inputs and 340 randomized inputs with no wrong value.

Two further sources of evidence arrived afterwards. First, three independent reviews of `StradFixed.wl` were written on 5 September 2026; they are kept under `src/code-review/` as `review-4`, `review-5` and `review-6` and are called R4, R5 and R6 below. Each contains a report, a proposed replacement implementation `StradImproved.wl` in the context `RadicalDenestImproved` ‘, a native regression suite, and independently executed Python/SymPy checks. Second, a probing session ran 136 further inputs through `StradFixed.wl`; it found no wrong value but fifteen denestable inputs that were left unchanged, grouped into five families A–E (`KNOWN_GAPS.md`, and Section 12.5 of unified-A).

1.2 What the reviews could and could not see

The three reviews differ in what they inspected, which matters for weighing their claims (Table 1). R4 pinned the repository commit and the Git blob of the reviewed file and read the complete source, the unified-A report and the literature guide. R6 read the complete source through its web rendering but could not pin a commit. R5 could read only the first part of the file, up to the beginning of the quadratic section: the whole of `DenestCore`, the multiplier scheduling and the discriminant generator were invisible to it, and it says so explicitly (`SOURCE_ACCESS.md`). None of the three could run a Wolfram kernel; every review states that its native tests and its proposed implementation were not executed. This document runs them (Section 6.6).

	R4	R5	R6
Title	Second audit of <code>StradFixed.wl</code>	Radical denesting after <code>StradFixed</code>	Radical denesting after the first repair
Source seen	complete, pinned commit 6d7739b	first ≈ 260 lines only; core unseen	complete, unpinned
Findings	F01–F14	F1–F10	F01–F16
Executed checks	1960 SymPy assertions	165 SymPy/lexical checks	33 named checks, 312 parameter cases
Wolfram execution	none	none	none
Proposed code	595 lines, FIFO queue, <code>DenestReport</code>	317 lines, GCD degree ≤ 2 by default	418 lines, fast paths for fourth roots and cubic-in-quadratic
Native tests supplied	37	31	43
Distinctive content	cap counterexamples; sentinel polynomial; norm-family and sign corrections of unified-A’s plan	winding counterexample for combine; three-valued equality	cap-7 counterexample; degree-gate countermodel; grammar for admissible output

Table 1: The three reviews of `StradFixed.wl` at a glance.

1.3 What this document adds

1. A finding-by-finding comparison of the three reviews, with an assessment of each finding against the actual source and, where the finding is about executable behaviour, against a kernel run (Section 2).
2. A consolidated catalogue of 23 issues B01–B23, each with its sources, its disposition in the rewrite and, where the rewrite deliberately departs from a review’s recommendation, the reason (Section 3).
3. The design of StradFixed2.wl: acceptance gate, session and budgets, option handling, grammar and cost, traversal, fast paths, Kummer factors and index reduction, multiplier queue, reporting (Section 4), with the mathematics of the new fast paths (Section 5).
4. Executed experiments: the unified-A battery and randomized families on the new version, the fifteen previously missed inputs, a regression suite of 144 tests translating the reviews’ suites, and the reviews’ own suites run against their own proposals (Section 6).
5. Remaining limitations, compatibility changes, and future work (Sections 7 and 8).

2 The three reviews

2.1 Where they agree

All three reviews reach the same principal conclusion: the first repair put the right invariant into the numerical core (candidates are certified against the original selected value, not against the principal root of its power), but the invariant is not enforced at every public boundary. The concrete symptom named by all three is the custom "Solver" option: the wrapper substitutes a callback’s answer for a marked radical, and its final equality check runs only when the *whole* input is an exact algebraic number. For a symbolic host such as $x + \sqrt{5 + 2\sqrt{6}}$ the check is skipped, so `Strad[x + Sqrt[5 + 2 Sqrt[6]]]`, `"Solver" -> (0 &)]` returns x . We confirmed this in the kernel (Section 2.4); it is the only wrong value that StradFixed.wl has been shown to return, and it requires the custom-solver option.

The reviews also agree on: the final `Simplify` of a symbolic host inherits the caller’s `$Assumptions` (R4 F07, R5 F2, R6 F02); the time and trial budgets are checked between trials only and restart in list elements and recursive calls (R4 F03, R5 F4, R6 F03); the `Strad` alias and the `FilterRules[{opts}, ...]` construction forward explicit rules but not the effective defaults of the entry point, so `SetOptions[Strad, ...]` is ineffective (R4 F02, R5 F5, R6 F06); the multiplier cap is not an invariant of `newMultipliers`, the product batch materializes 2^k combinations before truncation, and `"MultiplierCap" -> Infinity` makes a counting loop nonterminating (R4 F04–F05, R6 F04–F05); the guard in `combine` that permits $(FS)^p = F^p S^p$ for a positive aggregate F does not justify multiplying the exponents of the individually recorded factors (R4 F09, R5 F3, R6 F11); the numerical prefilter of `CertifiedEqualQ` is not a rigorous rejection test (R4 F08, R5 F6, R6 F10); polished candidates are not individually re-certified before the cheapest is chosen (R4 F10, R6 F09); the usage string of `RadicalCost` names four components while the function returns five, and radicals inside `Root` payloads are counted (R4 F11, R5 F7, R6 F12); and `ToRadicals` conversion is rolled back entirely if any opaque object remains (R4 F13, R5 F9, R6 F14).

2.2 Findings unique to one review

R4 F06 observes that `PolynomialQ[$Failed, x]` is `True`, so a failed `MinimalPolynomial` inside `RationalizeDenominator` passes the guard and the helper returns 0; we confirmed the predicate value in the kernel. R4 F14 corrects two statements of unified-A’s future-work section: the indirect

fourth-root criterion has the sign $-c(a^2 - b^2c)$, not $c(a^2 - b^2c)$, and the finite family of norm divisors is not a sufficient multiplier family, with the counterexample $\rho = 2 + \sqrt{3}$ of norm 1 that needs $m = 2$. Both corrections are right and are adopted here (Section 5). R4 F12 and R6 F07 analyse the visited set: it caches only the minimal-polynomial degree, whereas the work done by `tryMultiplier` depends on the current best degree, so a multiplier skipped as “visited” may never have had its GCD stage run. R6 F08 observes that the core can take its success exit because a certified candidate exists even when that candidate is not cheaper than the incumbent, so the search stops with an unchanged answer. The probing session found exactly this behaviour on the negative-radical fifth and sixth roots (the verbose log shows success: `\{1, 8\}` followed by an unchanged result), so R6 F08 is confirmed by an observed miss. R6 F13 notes the absence of fast paths for indirect fourth roots and cubic-in-quadratic roots; R5 F8 notes that `"Factor" -> False` does not disable the factorizer inside `normalizeRadicand`; R6 F12 and R4 F11 ask for an admissible output grammar so that a transcendental constant cannot pass as a depth-zero denesting.

2.3 Claims that we weigh differently

- **Reachability of the combine counterexample.** All three reviews present it as a helper-level proof gap and say that public reachability is unproved. We agree, and add that `FactorC` certifies its numeric output, so a wrong extraction is discarded, not returned. The rewrite removes the factorizer rather than repairing the guard, because its only observed effect on the battery was harmful (family C of the probes).
- **The numerical prefilter.** The reviews want it removed. It cannot cause a false acceptance, since acceptance requires an exact zero; its only risk is a false rejection, which loses a candidate. Removing it makes every orbit candidate go through `RootReduce`, which is the dominant cost of a trial. We keep it as a heuristic *rejection* step, guarded by the reported `Accuracy` of the difference and switchable with `"NumericPrefilter"`, and count its uses in the statistics (B08).
- **Shared trial budget.** R4, R5 and R6 share `"MaxTrials"` across all islands and list elements. In the rewrite the whole call is bounded by `"TimeBudget"`; `"MaxTrials"` bounds the trials spent on one island, because a shared count lets an inner island starve the outer one (this happened to the BFHT example during development, Section 4.9). The semantics are documented and tested; the reviews’ tests that assert a shared count were translated accordingly.
- **Inexact hosts.** R6 processes exact islands inside inexact trees; R4 and R5 leave such trees alone. The rewrite keeps the conservative policy of `StradFixed.wl`.
- **R5’s scope.** R5 did not see the core. Its findings about the wrapper are correct and were adopted; its silence about the core is not evidence about the core. Its own proposal restricts the GCD solver to degree at most two by default, which would miss most of the classical identities that need a cubic factor (Section 6.6).

2.4 Kernel reproductions of the reviews’ probes

The reviews supplied probes against `StradFixed.wl` that they could not execute. The relevant ones were run (log logs/probe_reviews_fixed.txt): `Strad[x + Sqrt[5 + 2 Sqrt[6]]]`, `"Solver" -> (0 &), "Factor" -> False` returns x (R4 F01, R5 F1, R6 F01, confirmed); `Block[{ $Assumptions = x > 0 }, Strad[Sqrt[x^2]]` returns x (R4 F07, R5 F2, confirmed); `PolynomialQ[$Failed, x]` is `True` (R4 F06, confirmed); the `cap-1` and `cap-7` helper calls of R4 and R6 return two and nine multipliers respectively (R4 F04, R6 F04, confirmed); `SetOptions[Strad, "TimeBudget" -> 0]` does not stop `Strad[Sqrt[5 + 2 Sqrt[6]]]` from denesting (R4 F02, confirmed).

3 The consolidated catalogue

Table 2 lists every issue addressed by the rewrite: the reviews’ findings merged where they describe the same defect, and the five probe families A–E. “Disposition” says what `StradFixed2.wl` does; the section references point to the design description.

Table 2: Consolidated catalogue of the issues addressed by `StradFixed2.wl`.

ID	Issue	Sources	Disposition
B01	A custom "Solver" result replaces a radical in a symbolic host without certification; the whole-input check is skipped for non-numeric hosts.	R4 F01, R5 F1, R6 F01	Fixed. One gate, accept, certifies every candidate against the island it replaces; a solver is a proposal (§4.1).
B02	The final <code>Simplify</code> of a symbolic host inherits ambient <code>\$Assumptions</code> ; the traversal domain is unspecified.	R4 F07, R5 F2, R6 F02	Fixed. Hosts are never simplified; <code>\$Assumptions = True</code> inside the session; only <code>List</code> , <code>Plus</code> , <code>Times</code> and <code>Power</code> are traversed (§4.6).
B03	Time and trial budgets are checked between trials only; one expensive trial, preprocessing, certification, lists and recursion escape them.	R4 F03, R5 F4, R6 F03; probe E	Fixed. Whole call in <code>TimeConstrained</code> and <code>MemoryConstrained</code> ; every kernel operation in <code>bounded[]</code> clipped to the remaining time; shared deadline (§4.2).
B04	Effective option defaults are not propagated: the alias forwards explicit rules, <code>coreOpts</code> filters explicit rules, <code>SetOptions</code> is ineffective.	R4 F02, R5 F5, R6 F06	Fixed. <code>OptionValue[head, rules, name]</code> resolves each entry point once; first explicit rule wins; one configuration association (§4.3).
B05	No option validation; " <code>MultiplierCap</code> " $\rightarrow$ <code>Infinity</code> makes the cap loop nonterminating.	R4 F05, R6 F05	Fixed. Types and ranges validated; unknown or invalid options return a <code>Failure</code> (§4.3).
B06	The cap of <code>newMultipliers</code> is leaked by a forced prime and by the union with all discriminant primes; the product batch builds 2^k combinations before truncation.	R4 F04, R6 F04	Fixed. Single admission closure; the cap counts every admission; proposals bounded by $4 \times \text{cap}$; prime powers streamed and a bounded prefix of mixed-radix products (§4.9).
B07	<code>PolynomialQ[\$Failed, x]</code> is <code>True</code> , so a failed minimal polynomial makes <code>RationalizeDenominator</code> return 0.	R4 F06	Fixed. <code>validPolynomialQ</code> rejects sentinels and constants; Horner inverse; certificate $d \cdot d^{-1} = 1$ before use (§4.10).
B08	<code>CertifiedEqualQ</code> : a fixed-tolerance numerical rejection, predicates outside the timed region, “unknown” collapsed into <code>False</code> .	R4 F08, R5 F6, R6 F10	Partly adopted. Three-valued <code>EqualityStatus</code> ; every step bounded; the prefilter kept as an Accuracy-guarded heuristic rejection, switchable and counted (§4.5).
B09	The positivity guard of <code>combine</code> does not justify termwise exponent multiplication (winding counterexample).	R4 F09, R5 F3, R6 F11	Fixed by removal. No custom factorizer; <code>Factor</code> is one certified proposal among others (§4.10).
B10	Polished candidates are not re-certified individually before the cheapest is picked.	R4 F10, R6 F09	Fixed. Every variant passes accept (§4.1).

ID	Issue	Sources	Disposition
B11	Cost: usage string says four components, function returns five; radicals inside Root payloads are counted; no admissible output grammar.	R4 F11, R5 F7, R6 F12	Fixed. Documented five-tuple with bit size; Root opaque; RadicalExpressionQ required of every output (§4.4).
B12	Visited set keyed by syntax and caching only the degree; degree gate depends on history; success exit without improvement.	R4 F12, R6 F07, R6 F08; probe A	Fixed. No degree gate, no history exit; improvement means an accepted cheaper certified candidate; patience rule (§4.9).
B13	ToRadicals rolled back entirely if any opaque object remains; inexact trees rejected wholesale.	R4 F13, R5 F9, R6 F14	Partly adopted. ToRadicals per island; inexact trees still left untouched by choice (§4.6).
B14	Missing exact fast paths: indirect fourth root, cubic-in-quadratic, Honsbeek, Gaussian square roots.	R6 F13, R4 §5, R5 §9	Fixed. All implemented, plus a multi-surd solver (§4.7, §5).
B15	Unified-A's future-work plan: wrong sign in the indirect criterion; norm divisors are not a sufficient multiplier family.	R4 F14	Acknowledged and corrected here (§5.1).
B16	"Factor" -> False does not disable factoring inside normalizeRadicand.	R5 F8	Fixed. One switch, one Factor proposal (§4.10).
B17	Marker wrapper: nesting, dependency order and marker removal need invariants; unresolved markers could leak.	R5 F10, R4 F13	Fixed by removal. Marker-free recursive walk (§4.6).
B18	Unchanged output conflates unsupported input, no candidate, exhaustion, timeout.	R6 F16, R4 §7	Fixed. DenestReport with status, limits, statistics, certificates (§4.11).
B19	Principal root of a negative radicand with index ≥ 5 is missed (8 inputs).	probe A	Fixed. Negative-radical separation $\rho^{p/q} = (-1)^{p/q} \rho ^{p/q}$ and linear factors of $X^q - \rho$ (§4.8).
B20	Composite index whose answer generates a larger field than the radicand is missed (4 inputs).	probe B	Fixed. Index reduction through linear factors of $X^k - \rho$, $k \mid q$, and recursion on equal-depth orbit candidates (§4.8).
B21	The factorizer extracts 16 from a radicand and hands the core a degree-12 problem.	probe C	Fixed by removal of factor extraction from radicands.
B22	Cancellation across summands (Landau's identity) is missed.	probe D	Fixed. Whole-island RootReduce and Simplify proposals for sums (§4.6).
B23	A five-surd square root exceeds the time budget inside one trial.	probe E	Fixed. Multi-surd solver and per-operation bounds (§4.7).

4 The design of StradFixed2.wl

The package lives in the context `RadicalDenest2` so that it can be loaded next to `RadicalDenest` for differential testing. Its public interface keeps the names of `StradFixed.wl` and adds `DenestReport`, `EqualityStatus` and `RadicalExpressionQ`:

- `Strad`, `DenestRadicals`, `DenestCore`, `DenestReport`;
- `EqualityStatus`, `CertifiedEqualQ`, `ExactAlgebraicQ`, `RadicalExpressionQ`;
- `RadicalDepth`, `RadicalCost`, `RationalizeDenominator`, `Factorc`.

The complete source is reproduced in Appendix A; the following subsections read it by responsibility.

4.1 One acceptance gate

Every candidate, whatever produced it, goes through accept:

```
accept[candidate_, target_, incumbent_, method_String] := Module[{},
  If[candidate === $Failed || candidate === target || ! RadicalExpressionQ[
    candidate] ||
    ! smallQ[candidate] || ! cheaperQ[candidate, incumbent], Return[incumbent
  ]];
  If[certify[candidate, target] === "Equal", ... candidate, incumbent]];
```

The target is the original island and never changes while the incumbent improves; this answers [B01](#) and [B10](#). The cheap syntactic tests come first (admissible grammar, size, strict cost improvement) so that the exact certificate is computed only for candidates that would be accepted. `acceptWithPolish` feeds a candidate and its `Simplify`, `Expand`, `Together` and rationalized variants through the same gate, so a variant is never trusted because its ancestor was certified. A user "Solver" is called inside `Catch` and bounded; its value is a proposal like any other (`genericProposals`).

4.2 Session, budgets and bounded operations

`run` opens one session: it Blocks the configuration, the deadline, the statistics, the limit flags, the memo of solved sub-problems and `$Assumptions = True`, and evaluates the passes inside `TimeConstrained[MemoryConstrained[...]]`. Every expensive kernel call (`MinimalPolynomial`, `PolynomialGCD`, `FactorList`, `Solve`, `RootReduce`, `Simplify`, `ToRadicals`, `Factor`, `Expand`, `N`) is wrapped in `bounded[body, kind]`, which gives it the smaller of its own allowance ("OperationTime" or "CertifyTime") and the remaining session time, silences messages, converts failures and timeouts into `$Failed`, and records them in the statistics and limit flags. The deadline is checked at every island, trial, orbit candidate and pass. A timeout of the whole region returns the last committed pass, never a partly rewritten host. This is [B03](#); the five-prime example that ran for 408 seconds under a 20-second budget in `StradFixed.wl` now finishes in about a second through the multi-surd fast path, and a hopeless search stops at the budget.

"MaxTrials" bounds the multiplier trials of one island (Section 4.9); the whole call is bounded by "TimeBudget". "TimeBudget" -> 0 is a documented no-op with status "Disabled". The standalone helpers `RationalizeDenominator`, `Factorc`, `EqualityStatus` and `CertifiedEqualQ` open a 20-second session of their own when called outside a denesting call.

4.3 Options

`resolveOptions[head, rules]` flattens the supplied rules, rejects non-rules and unknown names with a `Failure`, resolves every option through `OptionValue[head, rules, name]` (so `SetOptions` on the entry point actually called is honoured and the first explicit rule wins), and validates every value: Booleans must be `True` or `False`, counts nonnegative integers, limits positive integers, times finite nonnegative numbers, "Multipliers" Automatic or a list, "Solver" Automatic or a function. The resulting association is passed through the session; no inner function reconstructs defaults ([B04](#), [B05](#)). Each of `Strad`, `DenestRadicals`, `DenestCore` and `DenestReport` has its own option list, initially equal.

4.4 Grammar, depth and cost

`RadicalExpressionQ` defines the admissible output language: rationals, Gaussian rationals, and `Plus`, `Times` and rational `Power` combinations of them. `ExactAlgebraicQ` is the input grammar: the same plus exact `Root` and `AlgebraicNumber` objects. Exactness is decided by the grammar,

which is algebraic by construction; no `Element[... , Algebraics]` call is needed, so the predicate is cheap and deterministic. A transcendental constant, an inexact number or a symbol is not exact, so it can neither be replaced nor be accepted as a replacement (B11).

`RadicalDepth` counts nested rational-power nodes and treats `Root` and `AlgebraicNumber` as opaque. `RadicalCost` is the documented five-tuple

(`#opaque objects`, `depth`, `#radical nodes`, `LeafCount`, `bit size`),

where radical nodes inside opaque payloads are not counted and bit size is the total binary length of integer and rational atoms; `cheaperQ` is the lexicographic order via `Order`. The cost is not compositional (lowering a component's depth can raise the leaf count of a sum whose depth another component fixes), so run also requires every committed pass to be cheaper than the previous one.

4.5 Certification

`certify[a, b]` returns "Equal", "Different" or "Unknown". Both arguments must be exact; identical expressions are equal; then, if "NumericPrefilter" is on, the difference is evaluated to 40 digits and rejected as different only when the result is a number whose reported Accuracy is at least 25 and whose magnitude exceeds 10^{-20} ; then `RootReduce` of the difference (equal if it is the integer 0, different if it is a canonical nonzero algebraic object); then `PossibleZeroQ` with `Method -> "ExactAlgebraics"`. Everything runs inside bounded, and the statistics count equal, different, unknown and numerically rejected certificates. `CertifiedEqualQ` is True only for "Equal"; "Unknown" is reported as such by `EqualityStatus` (B08).

4.6 Traversal by congruence

`walk` descends through `List`, `Plus`, `Times` and `Power` with integer or rational exponent; any other head, including held expressions, is opaque and returned unchanged. An exact algebraic subtree is an *island* and is handed to `island`. A host containing inexact numbers is returned unchanged. No `Simplify` is ever applied to a host: a rewritten host equals the input by congruence, because every island replacement was certified against that island (B02, B13, B17).

Inside an island, `island` processes a rational-power node by `improveNumber`; with "AllLevels" `-> True` its radicand is first processed as an island of its own. A `Plus` or `Times` island first processes its components, then, if it still has depth at least two, the whole island, and finally, if anything changed, offers `RootReduce`, `Simplify`, `Together` and `Expand` of the rewritten island through the gate (`combineProposals`), which recombines $\frac{\sqrt{3}-1}{\sqrt{2}} + \frac{\sqrt{3}+1}{\sqrt{2}}$ into $\sqrt{6}$ and reduces Landau's cancellation sum to 0 (B22). `improveNumber` memoizes its result per island within the session, because the same sub-problem recurs through roots of unity, index reduction and repeated passes.

4.7 Fast paths

For a node $\rho^{p/q}$, `powerProposals` first tries the shape-specific recipes for the q -th root and raises each candidate to the integer power p : for $q = 2$ the direct and indirect quadratic formulas, Gaussian square roots, Honsbeek's square root of a sum of two cube roots, and the multi-surd solver; for $q = 3$ the cubic-in-quadratic trace-norm reconstruction; negative real radicands are handled by taking the recipe on $-\rho$ and multiplying by i or $(-1)^{1/3}$. The multi-surd solver applies to a radicand $a_0 + \sum a_i \sqrt{d_i}$ with two or more square roots of integers: it solves $(\sum_u x_u \sqrt{u})^2 = \rho$ over the rationals with `Solve[... , Rationals]`, first with unknowns indexed by 1 and the primes dividing the d_i , then with the d_i added; the five-prime example of family E is solved in this way in under a second. Section 5 states and proves the recipes (B14, B23).

4.8 Negative radicands, Kummer factors and index reduction

If the recipes fail, `negativeRadicandCandidate` handles a negative real radicand exactly: $\rho^{p/q} = (-1)^{p/q} (-\rho)^{p/q}$ because $\text{Arg } \rho = \pi$ lies in the principal range, and $(-\rho)^{p/q}$ is denested recursively. This is the whole of family A (B19): `StradFixed.wl` intersected $X^q - \rho$ with the minimal polynomial of the principal root, whose conjugates do not include the real root, and for $q \geq 5$ the remaining factor has degree four or more, which `Solve` returns as `Root` objects or nested surds.

`kummerCandidates` then factors $X^k - \rho$ over the radicals of ρ for every divisor $k \geq 2$ of q , largest first. The extension is given explicitly (`radicalExtension`) because `Extension -> Automatic` canonicalizes the radicals of ρ into `Root` objects and the linear factors then come out opaque; `radicalForms` additionally applies `ToRadicals` and `ToRadicals[RootReduce[...]]` to any opaque root. A linear factor $X - \gamma$ with $k = q$ gives the candidates $(\gamma \zeta_q^l)^p$; with $k < q$ it gives $\rho = \gamma^k$ and the sub-problems $(\gamma \zeta_k^j)^{1/(q/k)}$, which are denested recursively and multiplied by the (q/k) -th roots of unity. This is the index reduction of family B (B20): for Ramanujan's $\sqrt[6]{7\sqrt[3]{20} - 19}$ the factor $X^2 - \rho$ has the root $\gamma = 1 + \sqrt[3]{20} - \sqrt[3]{50}$ and $\sqrt[3]{\gamma}$ is denested by the cubic machinery. Roots of unity are written $(-1)^{2k/q}$, so they stay in the output grammar and are charged by the cost. Recursion is bounded by "MaxRecursion" and runs with a quarter of the trial budget.

4.9 The multiplier queue

`multiplierSearch` is the general engine of the original, restructured. The reduction index q is the LCM of the exponent denominators at maximal depth and $\rho = \text{Expand}[\text{target}^q]$. A FIFO queue is filled through one closure, `admit`, which caps admissions at "MultiplierCap" (counting seeds, forced multipliers and generated candidates alike), bounds proposal attempts at four times the cap, rejects inexact, zero and oversized values, and deduplicates by the expanded form. Seeds are 1, the complementary factors of the visible radicals of ρ , and 2, 3, 5; a supplied "Multipliers" list replaces them. A trial computes $\theta = m\rho$ without canonicalizing it, the minimal polynomial of $\theta^{1/q}$ (rejected above "MaxDegree"), the linear factors of $X^q - \theta$ over the radicals of θ , and the GCD of the minimal polynomial with $X^q - \theta$, whose roots are solved when its degree is at most "MaxSolveDegree"; every root, divided by $m^{1/q}$ and multiplied by each q -th root of unity, is offered through `acceptWithPolish`. A root that is certified equal to the target but not cheaper is denested recursively, which recovers the composite-index cases. Discriminant primes of a trial whose degree is the smallest seen so far generate at most 24 further multipliers per trial: the prime powers mp^e and a prefix of the mixed-radix products. The search stops when the incumbent has depth at most one, when the queue is exhausted, when "MaxTrials" trials have been spent on this island, or when "Patience" trials have passed since the last improvement (B06, B12).

The trial counter is per island. During development a shared counter let the search on the radicand of the BFHT example $\sqrt{16 - 2\sqrt{29} + 2\sqrt{55 - 10\sqrt{29}}}$, which is a sum and has no denesting of its own, exhaust the budget before the outer square root was tried. The multiplier search is now also restricted to radical nodes and products of radical nodes; sum islands receive the whole-island proposals only.

4.10 Helpers

`RationalizeDenominator` uses `Together`, a bounded `MinimalPolynomial` validated by `validPolynomialQ` (no sentinel, a polynomial of degree between 1 and "MaxDegree", rational coefficients, nonzero constant term), the Horner form of $d^{-1} = -(a_1 + a_2d + \dots + a_nd^{n-1})/a_0$, and the certificate `certify[den inv, 1] == "Equal"` before the numerator is multiplied (B07). The identity is local to the denominator, so a symbolic numerator is allowed. `Factorc` offers `Factor` of an exact input through `accept` and returns symbolic input unchanged (B09, B16). The recursive factor-list machinery of `StradFixed.wl` (`fullfactor`, `combine`, `normalizeRadicand`) is gone.

4.11 Reporting

`DenestReport` returns an association with "Result", "Status" ("Improved", "Unchanged", "Timeout", "MemoryLimit", "Disabled"), "Limits" (which thresholds were reached), "Statistics" (islands, trials, operations, timeouts, certificates by outcome, numeric rejections, multipliers proposed, admitted and duplicated, fast-path successes, passes), "Certificates" (the accepted replacements with their method), "InitialCost", "FinalCost", "ElapsedSeconds", "Options", an optional bounded "Trace", and "CompletenessClaim" $\rightarrow$ False (B18). Invalid options give a `Failure` object from every entry point.

5 The mathematics of the new fast paths

The results below are classical; they are stated in the form the code uses, with proofs short enough to check the branch conditions. Unified-A proves the direct quadratic formula and the branch algebra of the multiplier search; R4 and R6 give proofs of the indirect and cubic formulas that agree with the ones here.

5.1 Direct and indirect square roots

Let $a, b, c \in \mathbb{Q}$, $c > 0$ not a square, $b \neq 0$, and $\rho = a + b\sqrt{c} > 0$. Put $D = a^2 - b^2c$.

Proposition 5.1 (direct). *If $D = s^2$ with $s \in \mathbb{Q}_{\geq 0}$, then $a > 0$, $0 \leq s < a$, and $\sqrt{\rho} = \sqrt{(a+s)/2} + \text{sgn}(b)\sqrt{(a-s)/2}$.*

Proof. $D \geq 0$ gives $|a| \geq |b|\sqrt{c}$, and $\rho > 0$ then forces $a > 0$; $a^2 - s^2 = b^2c > 0$ gives $s < a$. With $u = (a+s)/2$, $v = (a-s)/2$ we have $u+v = a$, $uv = b^2c/4$, so $(\sqrt{u} + \text{sgn}(b)\sqrt{v})^2 = \rho$; the expression is positive because $u > v \geq 0$ (for $b < 0$, $s = 0$ would make c a rational square). $\square$

Proposition 5.2 (indirect). *If $D < 0$ and $e = \sqrt{b^2 - a^2/c} \in \mathbb{Q}$, then $b > 0$, $0 < e \leq b$, and $\sqrt{\rho} = c^{1/4}(\sqrt{(b+e)/2} + \text{sgn}(a)\sqrt{(b-e)/2})$. The condition is equivalent to $-cD$ being a rational square.*

Proof. $D < 0$ gives $|a| < |b|\sqrt{c}$ and $\rho > 0$ forces $b > 0$. The square of the bracket is $b + \text{sgn}(a)\sqrt{b^2 - e^2} = b + a/\sqrt{c}$, so multiplying by $\sqrt{c}$ gives ρ ; the bracket is positive since its first term is at least its second. Finally $-cD = c^2e^2$. $\square$

This is the sign that unified-A's future-work item stated wrongly (B15); the implementation uses $-cD$. For $\sqrt{4+3\sqrt{2}}$, $D = -2$ and $-cD = 4$, giving $2^{1/4}(1 + \sqrt{2})$.

5.2 Gaussian square roots

For $a, b \in \mathbb{Q}$ with $b \neq 0$ and $n = \sqrt{a^2 + b^2} \in \mathbb{Q}$, $\sqrt{a+bi} = \sqrt{(n+a)/2} + i\text{sgn}(b)\sqrt{(n-a)/2}$: the square is $a+bi$ and the real part is nonnegative with imaginary part of the sign of b , which is the principal branch. The kernel evaluates many such square roots itself; the recipe covers those it leaves alone.

5.3 Cube roots inside a real quadratic field

Let $c > 0$ be a nonsquare rational, $a, b \in \mathbb{Q}$, $b \neq 0$. A real cube root of $a + b\sqrt{c}$ of the form $A + B\sqrt{c}$ with $A, B \in \mathbb{Q}$ exists if and only if there are rational n, T with $n^3 = a^2 - b^2c$ and $T^3 - 3nT - 2a = 0$; then $T^2 \neq n$, $A = T/2$ and $B = b/(T^2 - n)$. Necessity: with $n = A^2 - cB^2$ and $T = 2A$, the norm and the rational part of the cube give the two equations, and the irrational part gives $b = B(T^2 - n)$. Sufficiency: squaring $T(T^2 - 3n) = 2a$ gives $(T^2 - 4n)(T^2 - n)^2 = 4b^2c > 0$, so $T^2 \neq n$ and

$cB^2 = A^2 - n$, after which both parts of $(A + B\sqrt{c})^3$ are a and b . The code finds n by an exact rational cube-root test and T among the rational roots of the cubic via `FactorList`, never by enumerating divisors, and certifies the reconstructed cube exactly. A negative radicand gives a real root that is not the principal power; the negative-radical separation and the roots-of-unity orbit select the principal value through the gate.

5.4 Honsbeek's square root of two cube roots

Let $A^3 = \alpha$, $B^3 = \beta$ be nonzero rationals with A, B real and $A + B > 0$, and $q = \beta/\alpha$. If $s \in \mathbb{Q}$ satisfies $s^4 + 4s^3 + 8qs - 4q = 0$ and $\beta - s^3\alpha > 0$, then

$$\sqrt{A + B} = \pm \frac{-s^2 A^2/2 + sAB + B^2}{\sqrt{\beta - s^3\alpha}},$$

with the sign making the value positive, because the polynomial identity $(-s^2/2 + su + u^2)^2 - (u^3 - s^3)(1 + u) = (s^4 + 4s^3 + 8u^3s - 4u^3)/4$ with $u = B/A$, multiplied by A^4 , gives $(-s^2 A^2/2 + sAB + B^2)^2 = (\beta - s^3\alpha)(A + B)$. For $A = \sqrt[3]{5}$, $B = -\sqrt[3]{4}$ one has $q = -4/5$, $s = -2$, $\beta - s^3\alpha = 36$, and the classical $\sqrt{\sqrt[3]{5} - \sqrt[3]{4}} = (\sqrt[3]{2} + \sqrt[3]{20} - \sqrt[3]{25})/3$; the reversed order gives $q = -5/4$, $s = 1$, denominator 9, the same value.

5.5 Multi-surd square roots

Let $\rho = a_0 + \sum_i a_i \sqrt{d_i}$ with rational coefficients and distinct squarefree integers $d_i > 1$, and let P be the set of primes dividing some d_i . If $\sqrt{\rho} = \sum_{u \in U} x_u \sqrt{u}$ with $x_u \in \mathbb{Q}$ and U a set of squarefree integers, then expanding the square and comparing coefficients over the basis $\{\sqrt{u}\}$ of the multiquadratic field gives a system of quadratic equations in the x_u with rational coefficients, and every rational solution gives a candidate whose square is ρ ; the sign is decided by the gate. The code tries $U = \{1\} \cup P$ and then $U = \{1\} \cup P \cup \{d_i\}$; it does not try every squarefree product of P , so a denesting whose surds are not among these is left to the multiplier search. For $(\sqrt{2} + \sqrt{3} + \sqrt{5} + \sqrt{7} + \sqrt{11})^2$ the first set suffices and `Solve` returns the two sign-related solutions in about a second.

5.6 Negative radicands and the linear factors of $X^q - \rho$

For a negative real ρ and a rational exponent p/q , $\rho^{p/q} = \exp(\frac{p}{q}(\log |\rho| + i\pi)) = |\rho|^{p/q}(-1)^{p/q}$, since $\text{Arg } \rho = \pi$ is in the principal range; this is exact for every p/q and needs no orbit. More generally, if $X^k - \rho$ has a root γ in the field generated by the radicals of ρ and $k \mid q$, then $\rho^{1/q}$ is one of $(\gamma \zeta_k^j)^{1/(q/k)} \zeta_{q/k}^l$, because $(\rho^{1/q})^{q/k}$ is a k -th root of ρ , hence $\gamma \zeta_k^j$ for some j ; the candidates are enumerated and the gate selects the principal one. Unified-A's plan proposed a finite family of multipliers derived from norm divisors as sufficient; R4 F14 refutes this with $\rho = 2 + \sqrt{3}$ (norm 1, multiplier 2 needed). The rewrite therefore treats the multiplier family as a heuristic and relies on the exact factorization steps above where they apply.

6 Experiments

6.1 Setup

All runs use the scripts in `harness/`, which load `src/corrected/StradFixed2.wl` and record every result exactly as unified-A did: `runCase` captures the value, the messages, the wall time, exact equality with the expected value by `RootReduce`, and the radical depth before and after. The transcripts are in `logs/`; the tables are produced by `harness/export_tables2.wl` from the recorded rows. One kernel ran at a time.

6.2 The unified-A battery

Tables 3–7 repeat the battery of unified-A (Section 5.7 there) on `StradFixed2.wl`, next to the depth and time recorded there for `StradFixed.wl`. Every output is exactly equal to its input and no kernel message was emitted. Of the 116 rows, 71 reduce the radical depth; 0 differ from the true value; 0 emit a message. The total wall time is 109 s, and the slowest row is L09 at 17.5 s. Compared with the first corrected version, the new version denests the same identities and additionally the negative-base roots of higher index; the depth-3 inputs A28–A31 reach the same depth-2 results; the non-denestable controls of group B are returned unchanged.

Table 3: Classical identities: first corrected version (Table 5.16 of unified-A) against `StradFixed2`. Depth is the syntactic radical depth before and after; every `StradFixed2` output is exactly equal to its input.

ID	Input	StradFixed depth	time (s)	StradFixed2 depth	time (s)	Output of StradFixed2
A01	<code>strad[Sqrt[3 + 2*Sqrt[2]]]</code>	2→1	0.103	2→1	0.438	$1 + \text{Sqrt}[2]$
A02	<code>strad[Sqrt[5 + 2*Sqrt[6]]]</code>	2→1	0.055	2→1	0.167	$\text{Sqrt}[2] + \text{Sqrt}[3]$
A03	<code>strad[Sqrt[2 + Sqrt[3]]]</code>	2→1	0.057	2→1	0.205	$(1 + \text{Sqrt}[3])/\text{Sqrt}[2]$
A04	<code>strad[Sqrt[11 + 6*Sqrt[2]]]</code>	2→1	0.030	2→1	0.144	$3 + \text{Sqrt}[2]$
A05	<code>strad[Sqrt[3 + Sqrt[5]]]</code>	2→1	0.060	2→1	0.391	$(1 + \text{Sqrt}[5])/\text{Sqrt}[2]$
A06	<code>strad[Sqrt[7 + 2*Sqrt[10]]]</code>	2→1	0.040	2→1	0.132	$\text{Sqrt}[2] + \text{Sqrt}[5]$
A07	<code>strad[Sqrt[6 + 2*Sqrt[2] + 2*Sqrt[3] + 2*Sqrt[6]]]</code>	2→1	0.093	2→1	0.716	$1 + \text{Sqrt}[2] + \text{Sqrt}[3]$
A08	<code>strad[Sqrt[12 + 2*Sqrt[6] + 2*Sqrt[14] + 2*Sqrt[21]]]</code>	2→1	0.231	2→1	0.290	$\text{Sqrt}[2] + \text{Sqrt}[3] + \text{Sqrt}[7]$
A09	<code>strad[(Sqrt[5] - 2)^(1/3)]</code>	2→1	0.045	2→1	0.098	$(-1 + \text{Sqrt}[5])/2$
A10	<code>strad[(2 + Sqrt[5])^(1/3)]</code>	2→1	0.046	2→1	0.083	$(1 + \text{Sqrt}[5])/2$
A11	<code>strad[(10 + 6*Sqrt[3])^(1/3)]</code>	2→1	0.066	2→1	0.068	$1 + \text{Sqrt}[3]$
A12	<code>strad[(2^(1/3) - 1)^(1/3)]</code>	2→1	0.337	2→1	1.200	$(1 - 2^(1/3) + 2^(2/3))/3^(2/3)$
A13	<code>strad[(5^(1/3) - 4^(1/3))^(1/3)]</code>	2→2	1.503	2→2	4.505	$(-2^(2/3) + 5^(1/3))^(1/3)$
A14	<code>strad[((11 + 5*Sqrt[5])/2)^(1/5)]</code>	2→1	0.045	2→1	0.121	$(1 + \text{Sqrt}[5])/2$
A15	<code>strad[Sqrt[1 + 2*2^(1/3) + 2^(2/3)]]</code>	2→1	0.019	2→1	0.072	$1 + 2^(1/3)$
A16	<code>strad[(17 + 12*Sqrt[2])^(1/4)]</code>	2→1	0.042	2→1	0.060	$1 + \text{Sqrt}[2]$
A17	<code>strad[(99 + 70*Sqrt[2])^(1/6)]</code>	2→1	0.044	2→1	0.127	$1 + \text{Sqrt}[2]$
A18	<code>strad[(3 + 2*Sqrt[2])^(3/2)]</code>	2→1	0.027	2→1	0.039	$7 + 5*\text{Sqrt}[2]$

ID	Input	StradFixed		StradFixed2		Output of StradFixed2
		depth	time (s)	depth	time (s)	
A19	Strad[(3 + 2*Sqrt[2])^(-2^(-1))]	2→1	0.049	2→1	0.058	-1 + Sqrt[2]
A20	Strad[1/Sqrt[3 + 2*Sqrt[2]]]	2→1	0.042	2→1	0.061	-1 + Sqrt[2]
A21	Strad[Sqrt[3 + 2*Sqrt[2]] + Sqrt[5 + 2*Sqrt[6]]]	2→1	0.043	2→1	0.128	1 + 2*Sqrt[2] + Sqrt[3]
A22	Strad[Sqrt[3 + 2*Sqrt[2]] * Sqrt[5 + 2*Sqrt[6]]]	2→1	0.094	2→1	0.264	2 + Sqrt[2] + Sqrt[3] + Sqrt[6]
A23	Strad[Sqrt[2 + Sqrt[3]] + Sqrt[2 - Sqrt[3]]]	2→1	0.058	2→1	0.221	Sqrt[6]
A24	Strad[Sqrt[-1 + 2*I*Sqrt[2]]]	2→1	0.049	2→1	0.155	1 + I*Sqrt[2]
A25	Strad[Sqrt[2 + 2*I*Sqrt[3]]]	2→1	0.071	2→1	0.049	I + Sqrt[3]
A26	Strad[Sqrt[-3 - 2*Sqrt[2]]]	2→1	0.028	2→1	0.051	I*(1 + Sqrt[2])
A27	Strad[(3 + 2*Sqrt[2])^(1/4)]	2→2	0.070	2→2	16.650	Sqrt[1 + Sqrt[2]]
A28	Strad[Sqrt[3 + 2*Sqrt[3 + 2*Sqrt[2]]], True]	3→2	0.090	3→2	1.492	Sqrt[5 + 2*Sqrt[2]]
A29	Strad[Sqrt[3 + 2*Sqrt[3 + 2*Sqrt[2]]]]	3→2	0.093	3→2	2.151	Sqrt[5 + 2*Sqrt[2]]
A30	Strad[Sqrt[16 - 2*Sqrt[29] + 2*Sqrt[55 - 10*Sqrt[29]]], True]	3→2	1.165	3→2	11.966	Sqrt[5] + Sqrt[11 - 2*Sqrt[29]]
A31	Strad[Sqrt[16 - 2*Sqrt[29] + 2*Sqrt[55 - 10*Sqrt[29]]]]	3→2	0.531	3→2	4.555	Sqrt[5] + Sqrt[11 - 2*Sqrt[29]]

Table 4: Non-denestable inputs (B) and branch counterexamples (C).

ID	Input	StradFixed		StradFixed2		Output of StradFixed2
		depth	time (s)	depth	time (s)	
B01	Strad[Sqrt[2 + Sqrt[2]]]	2→2	0.041	2→2	0.960	Sqrt[2 + Sqrt[2]]
B02	Strad[Sqrt[1 + Sqrt[2]]]	2→2	0.023	2→2	1.016	Sqrt[1 + Sqrt[2]]
B03	Strad[(1 + Sqrt[2])^(1/3)]	2→2	0.069	2→2	1.349	(1 + Sqrt[2])^(1/3)
B04	Strad[Sqrt[2]]	1→1	0.003	1→1	0.001	Sqrt[2]
B05	Strad[2]	0→0	0.001	0→0	0.002	2
B06	Strad[Sqrt[1 + I]]	1→1	0.003	1→1	0.002	Sqrt[1 + I]
B07	Strad[Root[#1^4 - 10*#1^2 + 1 & , 4]]	0→1	0.017	0→1	0.081	Sqrt[2] + Sqrt[3]
B08	Strad[Sqrt[Sqrt[2] + Sqrt[3]]]	2→2	0.080	2→2	1.666	Sqrt[Sqrt[2] + Sqrt[3]]
B09	Strad[(2 + 2*I*Sqrt[3])^(1/3)]	2→2	0.269	2→2	3.043	(2 + (2*I)*Sqrt[3])^(1/3)

ID	Input	StradFixed		StradFixed2		Output of StradFixed2
		depth	time (s)	depth	time (s)	
C01	Strad[-Sqrt[3 + 2*Sqrt[2]]]	2→1	0.027	2→1	0.051	-1 - Sqrt[2]
C02	Strad[(-1 + 2*I*Sqrt[2])^(3/2)]	2→1	0.043	2→1	0.091	-5 + I*Sqrt[2]
C03	Strad[Sqrt[-1 + 2*I*Sqrt[2]]*Sqrt[-2 + 2*I*Sqrt[3]]]	2→1	0.138	2→1	0.225	-((-I + Sqrt[2])*(-I + Sqrt[3]))
C04	Strad[(-3 - 2*Sqrt[2])^(3/2)]	2→1	0.054	2→1	0.093	-7*I - (5*I)*Sqrt[2]
C05	Strad[Sqrt[-2 - 2*Sqrt[2]]*Sqrt[-3 - 2*Sqrt[2]]]	2→2	0.050	2→2	3.448	-Sqrt[2*(7 + 5*Sqrt[2])]
C06	Strad[Sqrt[-3 - 2*Sqrt[2]]*Sqrt[-3 - 2*Sqrt[2]]]	1→1	0.009	1→1	0.002	-3 - 2*Sqrt[2]
C07	Strad[(-1 + 2*I*Sqrt[2])^(5/2)]	2→1	0.045	2→1	0.069	1 - (11*I)*Sqrt[2]
C08	Strad[(-5 - 2*Sqrt[6])^(3/2)]	2→1	0.091	2→1	0.088	(-11*I)*Sqrt[2] - (9*I)*Sqrt[3]

Table 5: Direct core calls with forced multipliers (D) and symbolic input (E).

ID	Input	StradFixed		StradFixed2		Output of StradFixed2
		depth	time (s)	depth	time (s)	
D01	DenestCore[-Sqrt[3 + 2*Sqrt[2]], "Multipliers" -> {1}]	2→1	0.029	2→1	0.030	-1 - Sqrt[2]
D02	DenestCore[Sqrt[3 + 2*Sqrt[2]], "Multipliers" -> {0}]	2→1	0.012	2→1	0.034	1 + Sqrt[2]
D03	DenestCore[Sqrt[3 + 2*Sqrt[2]], "Multipliers" -> {}]	2→1	0.006	2→1	0.024	1 + Sqrt[2]
D04	DenestCore[(-1 + 2*I*Sqrt[2])^(3/2), "Multipliers" -> {1}]	2→1	0.028	2→1	0.030	-5 + I*Sqrt[2]
D05	DenestCore[Sqrt[-1 + 2*I*Sqrt[2]]*Sqrt[-2 + 2*I*Sqrt[3]], "Multipliers" -> {1}]	2→1	0.089	2→1	0.224	-((-I + Sqrt[2])*(-I + Sqrt[3]))
D06	DenestCore[Sqrt[5 + 2*Sqrt[6]], "Multipliers" -> {1, 6 - 2*Sqrt[6] - 2*Sqrt[2] + 2*Sqrt[3]}]	2→1	0.016	2→1	0.020	Sqrt[2] + Sqrt[3]
D07	DenestCore[Sqrt[5 + 2*Sqrt[6]], "Multipliers" -> {6 - 2*Sqrt[6] - 2*Sqrt[2] + 2*Sqrt[3]}]	2→1	0.016	2→1	0.017	Sqrt[2] + Sqrt[3]
D08	DenestCore[Sqrt[3 + 2*Sqrt[3 + 2*Sqrt[2]]]]	3→3	0.114	3→2	1.341	Sqrt[5 + 2*Sqrt[2]]

ID	Input	StradFixed		StradFixed2		Output of StradFixed2
		depth	time (s)	depth	time (s)	
D09	DenestCore[2 + Sqrt[3]]	1→1	0.001	1→1	0.001	2 + Sqrt[3]
D10	DenestCore[Sqrt[3 + 2* Sqrt[2]], "Multipliers" -> {2.}]	2→1	0.013	2→1	0.014	1 + Sqrt[2]
D11	DenestCore[Sqrt[3 + 2* Sqrt[2]], "Multipliers" -> {Pi}]	2→1	0.013	2→1	0.018	1 + Sqrt[2]
D12	DenestCore[Sqrt[5 + 2* Sqrt[6]], "Multipliers" -> {1, Sqrt[5]}]	2→1	0.013	2→1	0.024	Sqrt[2] + Sqrt[3]
D13	DenestCore[Sqrt[3 + 2* Sqrt[2]], "Multipliers" -> {-1}]	2→1	0.012	2→1	0.022	1 + Sqrt[2]
D14	DenestCore[Sqrt[3 + 2* Sqrt[2]], "Multipliers" -> {1}]	2→1	0.012	2→1	0.018	1 + Sqrt[2]
D15	DenestCore[Sqrt[5 + 2* Sqrt[6]], "Multipliers" -> {1}]	2→1	0.019	2→1	0.026	Sqrt[2] + Sqrt[3]
E01	Strad[Sqrt[z^2]]	1→1	0.017	1→1	0.001	Sqrt[z^2]
E02	Strad[Sqrt[t + t^2]]	1→1	0.002	1→1	0.001	Sqrt[t + t^2]
E03	Strad[Sqrt[-x - y]]	1→1	0.002	1→1	0.001	Sqrt[-x - y]
E04	Strad[Sqrt[a + b]]	1→1	0.002	1→1	0.001	Sqrt[a + b]
E05	Strad[x]	0→0	0.000	0→0	0.001	x
E06	Strad[Sqrt[x]*Sqrt[3 + 2* Sqrt[2]]]	2→1	0.018	2→1	0.013	(1 + Sqrt[2])*Sqrt[x]
E07	Strad[f[1]]	0→0	0.001	0→0	0.001	f[1]
E08	Strad[f[Sqrt[3 + 2*Sqrt[2]]]]	2→1	0.014	2→2	0.001	f[Sqrt[3 + 2*Sqrt[2]]]
E09	Strad[RadicalDenest2'Private'mark[1]]	0→0	0.001	0→0	0.001	RadicalDenest2'Private'mark[1]

Table 6: Factorizer and rationalizer probes (J) and miscellaneous inputs (K).

ID	Input	StradFixed		StradFixed2		Output of StradFixed2
		depth	time (s)	depth	time (s)	
J01	Strad[Sqrt[-(Sqrt[2] + (-1)^(1/3))]]	2→2	0.480	2→2	1.670	Sqrt[-(-1)^(1/3) - Sqrt[2]]
J02	Strad[Sqrt[-(Sqrt[2] + (-1)^(1/3)) + 1]]	2→2	0.437	2→2	1.830	1 + Sqrt[-(-1)^(1/3) - Sqrt[2]]
J06	Strad[(-(Sqrt[2] + (-1)^(1/3)))^(1/3)]	2→2	1.826	2→2	2.284	(-(-1)^(1/3) - Sqrt[2])^(1/3)
J09	Factorc[Sqrt[-(Sqrt[2] + (-1)^(1/3))]]	2→2	0.002	2→2	0.001	Sqrt[-(-1)^(1/3) - Sqrt[2]]
J10	Factorc[Sqrt[-3 - 2*Sqrt[2]]]	2→2	0.001	2→2	0.001	I*Sqrt[3 + 2*Sqrt[2]]

ID	Input	StradFixed		StradFixed2		Output of StradFixed2
		depth	time (s)	depth	time (s)	
J11	Factorc[Sqrt[8 + 4*Sqrt[3]]]	2→2	0.006	2→2	0.007	2*Sqrt[2 + Sqrt[3]]
J12	Factorc[Sqrt[z^2]]	1→1	0.001	1→1	0.000	Sqrt[z^2]
J13	Factorc[Sqrt[-x - y]]	1→1	0.002	1→1	0.000	Sqrt[-x - y]
J14	Factorc[Sqrt[t + t^2]]	1→1	0.001	1→1	0.000	Sqrt[t + t^2]
J15	RationalizeDenominator[x/(1 + Sqrt[2])]	1→1	0.001	1→1	0.002	-x + Sqrt[2]*x
J16	RationalizeDenominator[1/(2 + 1/Sqrt[2])]	1→1	0.001	1→1	0.000	(1 + Sqrt[2])/2
J17	RationalizeDenominator[1/x]	0→0	0.000	0→0	0.000	x^(-1)
K01	Strad[Sqrt[28^(1/3) - 3]]	2→1	0.121	2→1	0.204	(-1 - 2^(2/3)*7^(1/3) + 2^(1/3)*7^(2/3))/3
K02	Strad[(1 + 2^(1/3))^3]	1→1	0.007	1→1	0.002	(1 + 2^(1/3))^3
K04	Strad[Strad[Sqrt[16 - 2*Sqrt[29] + 2*Sqrt[55 - 10*Sqrt[29]]], True], True]	2→2	1.762	2→2	9.961	Sqrt[5] + Sqrt[11 - 2*Sqrt[29]]
K09	Strad[Sqrt[5 + 2*Sqrt[6]]/Sqrt[3 + 2*Sqrt[2]]]	2→1	0.284	2→1	0.128	2 - Sqrt[2] - Sqrt[3] + Sqrt[6]
K10	Strad[Exp[I*(Pi/7)]*Sqrt[3 + 2*Sqrt[2]]]	2→1	0.107	2→1	0.017	(1 + Sqrt[2])*E^((I/7)*Pi)
K11	Strad[Sqrt[3 + 2*Sqrt[2]] + Pi]	2→1	0.035	2→1	0.019	1 + Sqrt[2] + Pi
K12	Strad[Sqrt[3. + 2*Sqrt[2]]]	0→0	0.000	0→0	0.001	2.414213562373095
K14	Strad[{Sqrt[3 + 2*Sqrt[2]], Sqrt[5 + 2*Sqrt[6]]}]	2→1	0.103	2→1	0.042	{1 + Sqrt[2], Sqrt[2] + Sqrt[3]}
K15	Strad[Sqrt[3 + 2*Sqrt[2]] == 1 + Sqrt[2]]	0→0	0.002	0→0	0.002	True
K17	Strad[Sqrt[(3 + 2*Sqrt[2])/(5 + 2*Sqrt[6])]]	2→1	0.206	2→1	0.135	-2 - Sqrt[2] + Sqrt[3] + Sqrt[6]
K19	Strad[Sqrt[Sqrt[2] + 1]^3]	2→2	0.065	2→2	0.879	(1 + Sqrt[2])^(3/2)
K20	Strad[Sqrt[2 + Sqrt[3]]*Sqrt[3 + 2*Sqrt[2]]]	2→1	0.148	2→1	0.197	((2 + Sqrt[2])*(1 + Sqrt[3]))/2
K21	Strad[Sqrt[3 + 2*Sqrt[2]], "Verbose" -> False, "MaxTrials" -> 1]	2→1	0.035	2→1	0.032	1 + Sqrt[2]
K22	Strad[Sqrt[5 + 2*Sqrt[6]], "Factor" -> False]	2→1	0.041	2→1	0.038	Sqrt[2] + Sqrt[3]

Table 7: Higher-order roots (L) and roots of negative bases (Q).

ID	Input	StradFixed		StradFixed2		Output of StradFixed2
		depth	time (s)	depth	time (s)	
L01	Strad[(239 + 169*Sqrt[2])^(1/7)]	2→1	0.067	2→1	0.066	1 + Sqrt[2]

ID	Input	StradFixed		StradFixed2		Output of StradFixed2
		depth	time (s)	depth	time (s)	
L02	Strad[(2 + Sqrt[2])^(1/5)]	2→2	0.430	2→2	1.349	(2 + Sqrt[2])^(1/5)
L03	Strad[(1 + Sqrt[2])^(1/7)]	2→2	0.773	2→2	2.023	(1 + Sqrt[2])^(1/7)
L04	Strad[Sqrt[17 + 2*Sqrt[6] + 2*Sqrt[10] + 2*Sqrt[14] + 2*Sqrt[15] + 2*Sqrt[21] + 2*Sqrt[35]]]	2→1	2.916	2→1	0.344	Sqrt[2] + Sqrt[3] + Sqrt[5] + Sqrt[7]
L05	Strad[Expand[(1 + Sqrt[2] + Sqrt[3])^3]^(1/3)]	2→1	0.118	2→1	0.135	1 + Sqrt[2] + Sqrt[3]
L06	Strad[Expand[(Sqrt[2] + Sqrt[3])^4]^(1/4)]	2→1	0.057	2→1	0.529	Sqrt[2] + Sqrt[3]
L07	Strad[Sqrt[1000003 + 2*Sqrt[999983]]]	2→2	0.212	2→2	4.292	Sqrt[1000003 + 2*Sqrt[999983]]
L08	Strad[Sqrt[2 + Sqrt[3] + Sqrt[5]]]	2→2	0.376	2→2	2.033	Sqrt[2 + Sqrt[3] + Sqrt[5]]
L09	Strad[(5 + 2*Sqrt[6])^(1/6)]	2→2	1.554	2→2	17.495	(5 + 2*Sqrt[6])^(1/6)
L10	Strad[(11*Sqrt[2] + 9*Sqrt[3])^(1/3)]	2→1	0.054	2→1	0.048	Sqrt[2] + Sqrt[3]
L11	Strad[Expand[(2^(1/3) + 3^(1/3))^3]^(1/3)]	2→1	0.266	2→1	0.094	2^(1/3) + 3^(1/3)
L12	Strad[Sqrt[Expand[(1 + 2^(1/3) + 3^(1/3))^2]]]	2→1	0.058	2→1	0.070	1 + 2^(1/3) + 3^(1/3)
L13	Strad[Expand[(1 + 2^(1/5))^5]^(1/5)]	2→1	0.055	2→1	0.081	1 + 2^(1/5)
L14	Strad[Sqrt[5 + 2*Sqrt[6]]^(1/5)]	2→2	6.605	2→2	1.838	(5 + 2*Sqrt[6])^(1/10)
Q01	Strad[Expand[(-1 - Sqrt[2])^3]^(1/3)]	2→1	0.401	2→1	0.066	(-1)^(1/3)*(1 + Sqrt[2])
Q02	Strad[Expand[(1 - Sqrt[2])^3]^(1/3)]	2→1	0.366	2→1	0.062	(-1)^(1/3)*(-1 + Sqrt[2])
Q07	Strad[Expand[(2 - 3*Sqrt[2])^3]^(1/3)]	2→1	0.475	2→1	0.068	(-1)^(1/3)*(-2 + 3*Sqrt[2])
Q08	Strad[(-7 - 5*Sqrt[2])^(1/3)]	2→1	0.212	2→1	0.061	(-1)^(1/3)*(1 + Sqrt[2])

6.3 Randomized families

The ten structured families of unified-A (Section 5.3 there), with the same seed SeedRandom[20260905] and the same 60-second limit per input, were run on the new version (Table 8). All 340 inputs were processed in 81 s; 0 results differ from the true value; every constructed identity (families F1, F2, F4–F10) is denested, and the generic family F3 yields the same six denestings as before. The F3 controls are where the new version spends its time: a non-denestable square root runs the multiplier search to its patience or trial limit.

6.4 The fifteen previously missed inputs

The four probe scripts of the probing session were rerun against StradFixed2.wl (the scripts probe_gaps2_1.wl to probe_gaps2_4.wl in harness/, transcripts of the same names in logs/). Table 9 lists the fifteen inputs of KNOWN_GAPS.md with the new results: every one is denested to

Family	n	StradFixed			StradFixed2			StradFixed2 time (s)	
		denested	unchanged	wrong	denested	unchanged	wrong	total	max
F1	40	40	0	0	40	0	0	0.992	0.073
F2	30	30	0	0	30	0	0	1.806	0.107
F3	40	6	34	0	6	34	0	56.917	3.483
F4	40	40	0	0	40	0	0	1.516	0.101
F5	40	40	0	0	40	0	0	1.549	0.052
F6	40	40	0	0	40	0	0	3.760	0.123
F7	25	25	0	0	25	0	0	1.396	0.081
F8	25	25	0	0	25	0	0	4.786	0.489
F9	30	30	0	0	30	0	0	1.283	0.074
F10	30	30	0	0	30	0	0	7.416	0.490
Total	340	306	34	0	306	34	0	81.420	3.483

Table 8: Randomized families: first corrected version (from unified-A) against StradFixed2. “unchanged” includes equal rewritten outputs. F1 $\sqrt{(p + q\sqrt{c})^2}$; F2 $((p + q\sqrt{c})^3)^{1/3}$; F3 $\sqrt{a + b\sqrt{c}}$ with random a ; F4 $\sqrt{(p + qi\sqrt{c})^2}$; F5 $((p + qi\sqrt{c})^2)^{3/2}$; F6 $\sqrt{u^2\sqrt{v^2}}$ with complex u, v ; F7 $((p + q\sqrt{c})^4)^{1/4}$; F8 $\sqrt{(p + q\sqrt{c} + r\sqrt{d})^2}$; F9 $((p + q\sqrt{c})^2)^{3/2}$, real; F10 $\sqrt{-u^2\sqrt{-v^2}}$, real.

the expected depth, the slowest (Landau’s cancellation sum, which first searches each summand and then reduces the whole to 0) in about eight seconds. The controls and kernel-evaluated inputs of the probing session behave as before. The fourth probe script, whose M series had never been executed because the probing session was stopped, exposed one further gap during this work: islands of the form (sum of radicals) ^{n} with an integer exponent, such as $1/(\sqrt{2} + \sqrt{3} + 2\sqrt{2})$ and $(\sqrt{2} - \sqrt{3} + \sqrt{2} + \sqrt{3})^2$, were not descended. The final version processes the base of an integer power as an island, returns $(1 + 2\sqrt{2})^{-1}$ and 6 for these two, and the suite contains both.

Table 9: The fifteen inputs that StradFixed.wl left unchanged, on StradFixed2.wl. Every result is exactly equal to the input and has the expected depth.

Fam.	ID	Input	Result	Time (s)
A	C05	$(41 - 29\sqrt{2})^{1/5}$	$-(-1)^{1/5} + (-1)^{1/5}\sqrt{2}$	0.24
A	C14	$(-99 - 70\sqrt{2})^{1/6}$	$(-1)^{1/6} + (-1)^{1/6}\sqrt{2}$	0.25
A	S11	$(-21 - 15\cdot 2^{1/3} - 12\cdot 2^{2/3})^{1/5}$	$(-1)^{1/5} + (-1)^{1/5}\cdot 2^{1/3}$	0.22
A	S12	$(-239 - 169\sqrt{2})^{1/7}$	$(-1)^{1/7} + (-1)^{1/7}\sqrt{2}$	0.20
A	S13	$(76 - 44\sqrt{3})^{1/5}$	$-(-1)^{1/5} + (-1)^{1/5}\sqrt{3}$	0.12
A	S14	$(682 - 305\sqrt{5})^{1/5}$	$-2\cdot(-1)^{1/5} + (-1)^{1/5}\sqrt{5}$	0.04
A	S15	$(1393 - 985\sqrt{2})^{1/9}$	$-(-1)^{1/9} + (-1)^{1/9}\sqrt{2}$	0.12
A	S20	$(109\sqrt{2} - 89\sqrt{3})^{1/5}$	$-((-1)^{1/5}\sqrt{2}) + (-1)^{1/5}\sqrt{3}$	0.17
B	R03	$(-19 + 7\cdot 2^{2/3}\cdot 5^{1/3})^{1/6}$	$-(2/3)^{1/3} + (5/3)^{1/3}$	0.46
B	P02	$(133 + 57\cdot 2^{2/3}\cdot 3^{1/3} + 48\cdot 2^{1/3}\cdot 3^{2/3})^{1/6}$	$2^{1/3} + 3^{1/3}$	0.09
B	S10	$(229 + 87\cdot 2^{2/3}\cdot 5^{1/3} + 60\cdot 2^{1/3}\cdot 5^{2/3})^{1/6}$	$2^{1/3} + 5^{1/3}$	0.07

Fam.	ID	Input	Result	Time (s)
B	S04	$(2555 + 1116 \cdot 2^{(2/3)} \cdot 3^{(1/3)} + 981 \cdot 2^{(1/3)} \cdot 3^{(2/3)})^{(1/9)}$	$2^{(1/3)} + 3^{(1/3)}$	0.09
C	S05	$(1296 + 880 \cdot \text{Sqrt}[2] + 720 \cdot \text{Sqrt}[3] + 528 \cdot \text{Sqrt}[6])^{(1/6)}$	$1 + \text{Sqrt}[2] + \text{Sqrt}[3]$	0.06
D	Q21	$\text{Sqrt}[1 + \text{Sqrt}[3]] + \text{Sqrt}[3 + 3 \cdot \text{Sqrt}[3]] - \text{Sqrt}[10 + 6 \cdot \text{Sqrt}[3]]$	0	2.73
E	P13	$\text{Sqrt}[28 + 2 \cdot \text{Sqrt}[6] + 2 \cdot \text{Sqrt}[10] + 2 \cdot \text{Sqrt}[14] + 2 \cdot \text{Sqrt}[15] + 2 \cdot \text{Sqrt}[21] + 2 \cdot \text{Sqrt}[22] + 2 \cdot \text{Sqrt}[33] + 2 \cdot \text{Sqrt}[35] + 2 \cdot \text{Sqrt}[55] + 2 \cdot \text{Sqrt}[77]]$	$\text{Sqrt}[2] + \text{Sqrt}[3] + \text{Sqrt}[5] + \text{Sqrt}[7] + \text{Sqrt}[11]$	0.58

6.5 The regression suite

tests/StradFixed2.wlt contains 144 VerificationTests: the 37 tests of R4, the 31 of R5 and the 43 of R6 translated to the new interface, the fifteen inputs of the probing session, and contracts kept from the unified-A battery. The translation replaces the context, maps option names ("MaxRootOrder" to "MaxRootIndex", "MaxRecords" to "MaxTraceEntries", "DetailedResult" to DenestReport, report keys to those of Section 4.11), drops options that have no counterpart ("UseDiscriminants", "StageTime", "MaxBytes", "MaxCandidates"), and adjusts the two tests that assert a trial count shared across a list to the per-island semantics. Two of R6's tests were inverted deliberately: R6 expects exact islands inside a list with an inexact element to be processed, and StradFixed2 documents the opposite policy. tests/run_tests.wls runs the suite in a fresh kernel: 144 of 144 tests succeed and 0 fail (log regression_suite.txt).

6.6 The reviews' own suites against their own proposals

None of the reviews could execute its proposed StradImproved.wl. harness/run_review_suite.wl loads each proposal in a fresh kernel and runs the review's own suite unchanged (logs review4_suite.txt, review5_suite.txt, review6_suite.txt). R4: 37 of 37 tests pass; failures: none. R5: 30 of 31 tests pass; failures: conjugates-distinguished (actual HoldForm["Unknown"]). R6: 43 of 43 tests pass; failures: none. All three proposals load without messages in Wolfram 15.0.1, which is itself a result the reviews could not establish. R5's single failure is in its own equality classifier: EqualityStatus[Sqrt[2], -Sqrt[2]] returns "Unknown" because RootReduce[2 Sqrt[2]] is 2 Sqrt[2], which its canonicalNonzeroQ does not recognize as a nonzero algebraic number; the same classifier would report "Unknown" for most unequal radical pairs, so the proposal's rejections rest on the cost test rather than on a proved inequality. The suites are otherwise modest in coverage: they exercise the classical two-surd identities, the interface boundaries and the option validation, not the higher-index or multi-surd identities of the batteries above; R5's proposal, whose GCD solver is restricted to degree two by default, is not tested on any input that needs a cubic factor.

6.7 Timing

The classical identities take between 0.02 and 0.4 seconds each (Table 3), about the same as the first corrected version. The cost moved to the non-denestable inputs: the first version returned them after a short multiplier stack was exhausted, the new version runs the bounded search to its patience or trial limit and, for composite indices, recurses into sub-problems, so an unchanged answer typically costs 1–4 seconds and the slowest row of the battery (I09) costs 17.5 seconds. Every run stayed far below its "TimeBudget" of 120 seconds; the reduced default "MaxTrials" of 120, the patience

rule, the memo of solved sub-problems and the cheap multiplier key (an expanded form instead of RootReduce) were each necessary to reach this.

7 Compatibility changes and remaining limitations

- **Option semantics.** "MaxTrials" is per island (StradFixed.wl: per core call, which was also per island in practice); "TimeBudget" bounds the whole call (previously the core loop only); "CertifyTime" and the new "OperationTime" bound single kernel operations; new options "MaxRootIndex", "MaxDegree", "MaxSolveDegree", "MaxLeafCount", "MaxPasses", "MaxRecursion", "Patience", "NumericPrefilter", "Trace", "MaxTraceEntries", "MemoryBudget". Default "MaxTrials" is 120 (was 400): a trial now includes a factorization over the extension, and the patience rule and fast paths make long searches rare. Invalid options return Failure instead of being ignored.
- **Behaviour.** DenestCore runs the fast paths as well as the multiplier search, so DenestCore[Sqrt[3 + 2 Sqrt[2]], "Multipliers" -> {0}] now denests where StradFixed.wl returned the input. Factorc no longer reproduces the custom factorizer's repertoire on symbolic input. Output forms can differ (for example $2^{1/4} + 2^{3/4}$ for $2^{1/4}(1 + \sqrt{2})$); the cost order, not a normal form, decides between equal outputs.
- **Completeness.** There is none. An unchanged result means that the fast paths, the Kummer factors, the whole-island proposals and a bounded multiplier search found nothing cheaper. The multiplier family is heuristic (B15); the multi-surd solver tries two unknown sets, not all squarefree products; the Honsbeek recipe requires exactly two real cube-root terms; sums whose denesting needs a joint transformation other than RootReduce or Simplify are still treated component by component.
- **Certification.** The numeric prefilter is a heuristic rejection; a genuine identity with catastrophic cancellation at 40 digits would be rejected and the candidate lost, never accepted wrongly. RootReduce remains the trusted oracle; the certificates in DenestReport are recheckable kernel records, not independent proof objects.
- **Resources.** TimeConstrained and MemoryConstrained are cooperative kernel controls; a callback that protects itself from interrupts is not contained. Argument evaluation before the call and the assembly of the report are outside the timed region.
- **Cost.** Typical classical identities take 0.02–0.4 seconds; non-denestable inputs take 1–4 seconds because the multiplier search runs to its budget; the slowest case of the battery, $(5 + 2\sqrt{6})^{1/6}$, spends about half a minute on recursive sub-problems before returning the input.

8 Future work

1. **Field-theoretic multiplier generation.** Replace the discriminant heuristic by the classes of $K^\times / (K^\times)^q$ for the relative extension actually at hand, as R4 recommends, with an explicit statement of which multipliers the family covers.
2. **Multiquadratic bases.** Represent square-root sums over a basis of independent square classes and use the character projections of R4 Section 8.2 for dependency-aware joint denesting of sums, which would subsume the multi-surd solver and Landau's cancellation identity.
3. **Higher odd indices in quadratic fields.** Extend the trace-norm recipe with the Dickson polynomials $D_k(T, n)$ of the literature guide, with the branch and denominator checks the cubic case already has.

4. **A Pareto frontier of certified alternatives** per subtree, so that a shallower but longer form and a deeper but shorter form can both survive until the containing expression decides.
5. **Independent certificates.** Record defining polynomials, field maps and isolating intervals so that an accepted identity can be rechecked without RootReduce.
6. **Cheaper trials.** Cache minimal polynomials and factorizations across sessions keyed by canonical values, with a status that distinguishes a timeout from a result.

A Source of StradFixed2.wl

```

1  (* ::Package:: *)
2
3  (* StradFixed2.wl -- second corrected version of src/original/Strad.wl.
4
5     Successor of StradFixed.wl (RadicalDenest'). It answers the three reviews of
6     the first corrected version (src/code-review/review-4, review-5, review-6)
7     and the probing session recorded in KNOWN_GAPS.md. The report in
8     src/code-review/unified-B explains every change.
9
10    Contract (for an exact algebraic input E and every exact algebraic island of
11    a symbolic host):
12    * every replaced island is certified equal to the island it replaces by
13    exact algebra, whatever produced the candidate (fast path, multiplier
14    search, ToRadicals, Simplify, Factor, or a user "Solver");
15    * an accepted replacement is strictly cheaper under RadicalCost, or the
16    island is returned unchanged;
17    * the whole call runs inside one TimeConstrained/MemoryConstrained region
18    ("TimeBudget", "MemoryBudget") whose deadline is shared by all islands,
19    list elements, passes and recursive sub-problems; every expensive kernel
20    operation runs inside its own bounded region; "MaxTrials" bounds the
21    multiplier trials spent on any one island;
22    * symbolic hosts are never simplified; ambient $Assumptions are ignored;
23    * invalid or unknown options produce a Failure, never a silent default.
24    There is no completeness or minimum-depth guarantee: an unchanged result
25    means that the enabled bounded methods found nothing cheaper.
26
27    Context: RadicalDenest2'. The package can be loaded next to RadicalDenest'
28    (StradFixed.wl) for differential testing. Author: analysis session of
29    5 September 2026. *)
30
31  BeginPackage["RadicalDenest2'"];
32
33  Strad::usage = "Strad[expr, opts] denests the exact algebraic radicals occurring in expr and
34    returns an expression certified equal to expr. Strad[expr, True, opts] also processes inner
35    nesting levels and repeats passes while they improve the result.";
36  DenestRadicals::usage = "DenestRadicals[expr, opts] is the option-driven form of Strad with its
37    own option defaults.";
38  DenestCore::usage = "DenestCore[problem, opts] denests one exact algebraic number without
39    traversing a symbolic host.";
40  DenestReport::usage = "DenestReport[expr, opts] returns an Association with the result (\
41    Result\), \Status\, \Limits\, \Statistics\, \Certificates\, \ElapsedSeconds\, \
42    Options\" and an optional bounded \Trace\.";
43  EqualityStatus::usage = "EqualityStatus[a, b] is \Equal\", \Different\" or \Unknown\" for
44    exact algebraic numbers a and b, decided by exact algebra within a time limit.";
45  CertifiedEqualQ::usage = "CertifiedEqualQ[a, b] is True only when EqualityStatus[a, b] is \
46    Equal\.";
47  ExactAlgebraicQ::usage = "ExactAlgebraicQ[e] is True when e is an explicit exact algebraic
48    number: rationals, Gaussian rationals, Root and AlgebraicNumber objects, and Plus, Times
49    and rational Power combinations of them.";
50  RadicalExpressionQ::usage = "RadicalExpressionQ[e] is True when e is an explicit radical
51    expression: rationals, Gaussian rationals and Plus, Times and rational Power combinations
52    of them. Root and AlgebraicNumber objects are not radical expressions.";
53  RadicalDepth::usage = "RadicalDepth[e] is the maximal number of nested rational-power nodes on
54    a path of e. Root and AlgebraicNumber objects are opaque and count as depth 0.";
55  RadicalCost::usage = "RadicalCost[e] is the lexicographic cost {#Root and AlgebraicNumber
56    objects, radical depth, #rational-power nodes, LeafCount, total bit size of integer and
57    rational atoms} used to decide whether a rewrite is an improvement.";

```

```

43 RationalizeDenominator::usage = "RationalizeDenominator[expr] rewrites expr with a rational
    denominator using a certified polynomial inverse of the exact algebraic denominator. It
    need not reduce RadicalCost.";
44 Factorc::usage = "Factorc[expr] offers Factor of an exact algebraic expr as a certified
    proposal and returns it only when it is certified equal and cheaper. Symbolic input is
    returned unchanged.";
45
46 Options[Strad] = {
47   "AllLevels" -> False, "Verbose" -> False, "Trace" -> False,
48   "Multipliers" -> Automatic, "Solver" -> Automatic, "Factor" -> True,
49   "MaxTrials" -> 120, "TimeBudget" -> 120, "OperationTime" -> 30,
50   "CertifyTime" -> 20, "MemoryBudget" -> 1073741824,
51   "MultiplierCap" -> 1000, "MaxRootIndex" -> 32, "MaxDegree" -> 64,
52   "MaxSolveDegree" -> 4, "MaxLeafCount" -> 20000, "MaxPasses" -> 4,
53   "MaxRecursion" -> 3, "Patience" -> 25, "NumericPrefilter" -> True, "MaxTraceEntries" -> 200};
54 Options[DenestRadicals] = Options[Strad];
55 Options[DenestCore] = Options[Strad];
56 Options[DenestReport] = Options[Strad];
57
58 Begin["Private"];
59
60 (* ----- *)
61 (* session state (dynamically scoped by run[]), statistics, tracing *)
62 (* ----- *)
63
64 $active = False;
65 $cfg = Association[Options[Strad]];
66 $deadline = Infinity;
67 $stats = <|>; $limits = <|>; $trace = {}; $records = {}; $memo = <|>;
68 $recursion = 0;
69 $x = Unique["RadicalDenest2Private"x"];
70
71 newStats[] := <|"Islands" -> 0, "Trials" -> 0, "Operations" -> 0,
72   "OperationTimeouts" -> 0, "OperationFailures" -> 0,
73   "Certificates" -> 0, "CertificatesEqual" -> 0, "CertificatesDifferent" -> 0,
74   "CertificatesUnknown" -> 0, "NumericRejections" -> 0,
75   "CandidatesAccepted" -> 0, "MultipliersProposed" -> 0,
76   "MultipliersAdmitted" -> 0, "DuplicateMultipliers" -> 0,
77   "FastPathAccepted" -> 0, "PassesCompleted" -> 0|>;
78
79 bump[key_String] := AssociateTo[$stats, key -> (Lookup[$stats, key, 0] + 1)];
80 limitHit[key_String] := AssociateTo[$limits, key -> True];
81 remaining[] := $deadline - AbsoluteTime[];
82 expiredQ[] := If[remaining[] <= 0, limitHit["TimeBudget"]; True, False];
83
84 log[args___] := If[TrueQ[$cfg["Verbose"]], Print["RadicalDenest2 ", args]];
85 trace[tag_String, data_<|>] := If[TrueQ[$cfg["Trace"]], &&
86   Length[$trace] < $cfg["MaxTraceEntries"],
87   AppendTo[$trace, <|"Event" -> tag, "Data" -> data|>]];
88
89 (* Every expensive kernel operation runs inside bounded[...]: its own time
90    limit, clipped to the remaining session time, with messages silenced and
91    failures turned into $Failed. *)
92 SetAttributes[bounded, HoldAll];
93 bounded[body_, kind_< "Operation"] := Module[{seconds, value},
94   seconds = Min[remaining[], If[kind == "Certificate", $cfg["CertifyTime"], $cfg["
95     OperationTime"]]];
96   If[seconds <= 0,
97     limitHit[If[remaining[] <= 0, "TimeBudget", kind <> "Time"]]; Return[$Failed];
98     bump["Operations"];
99     value = Quiet[Check[TimeConstrained[body, seconds, operationTimedOut], operationFailed]];
100     Which[
101       value == operationTimedOut,
102         bump["OperationTimeouts"]; trace["OperationTimeout"];
103         limitHit[If[kind == "Certificate", "CertifyTime", "OperationTime"]];
104         If[remaining[] <= 0, limitHit["TimeBudget"]]; $Failed,
105       value == operationFailed || value == $Failed || value == $Aborted,
106         bump["OperationFailures"]; $Failed,
107       True, value];
108   (* ----- *)

```

```

109 (* option resolution and validation *)
110 (* ----- *)
111
112 finiteNonnegativeQ[v_] := MatchQ[v, _Integer | _Rational | _Real] && TrueQ[0 <= v < Infinity];
113 nonnegativeIntegerQ[v_] := IntegerQ[v] && v >= 0;
114 positiveIntegerQ[v_] := IntegerQ[v] && v >= 1;
115 booleanQ[v_] := v === True || v === False;
116
117 flattenRules[items_List] := Flatten[Replace[items, l_List -> flattenRules[l], {1}], 1];
118
119 resolveOptions[head_Symbol, raw_List] := Module[{rules, names, unknown, cfg, bad},
120   rules = flattenRules[raw];
121   If[! AllTrue[rules, MatchQ[#, _Rule | _RuleDelayed] &],
122     Return[Failure["InvalidOption", <|"MessageTemplate" -> "Options must be rules.", "Rules" ->
123       rules|>]]];
123   names = First /@ Options[head];
124   unknown = Complement[First /@ rules, names];
125   If[unknown != {},
126     Return[Failure["UnknownOption", <|"MessageTemplate" -> "Unknown option(s): 'Keys'.", "Keys"
127       -> unknown|>]]];
127   (* effective defaults of the head actually called, first explicit rule wins *)
128   cfg = Association[Table[With[{name = key}, name -> OptionValue[head, rules, name]], {key,
129     names}]];
129   bad = Join[
130     Select[{"AllLevels", "Verbose", "Trace", "Factor", "NumericPrefilter"}, ! booleanQ[cfg[#]]
131       &],
131     Select[{"MaxTrials", "MultiplierCap", "MaxTraceEntries", "MaxRecursion", "Patience"}, !
132       nonnegativeIntegerQ[cfg[#]] &],
132     Select[{"MemoryBudget", "MaxRootIndex", "MaxDegree", "MaxSolveDegree", "MaxLeafCount", "
133       MaxPasses"}, ! positiveIntegerQ[cfg[#]] &],
133     Select[{"TimeBudget", "OperationTime", "CertifyTime"}, ! finiteNonnegativeQ[cfg[#]] &]];
134   If[! (cfg["Multipliers"] === Automatic || ListQ[cfg["Multipliers"]]), AppendTo[bad, "
135     Multipliers"]];
135   If[! (cfg["Solver"] === Automatic || MatchQ[cfg["Solver"], _Function | _Symbol]), AppendTo[
136     bad, "Solver"]];
136   If[cfg["MaxSolveDegree"] > 4, AppendTo[bad, "MaxSolveDegree"]];
137   If[bad != {},
138     Return[Failure["InvalidOption", <|"MessageTemplate" -> "Invalid value(s) for 'Keys'; limits
139       must be finite and of the documented type.", "Keys" -> bad|>]]];
139   cfg];
140
141 (* ----- *)
142 (* grammar, depth, cost *)
143 (* ----- *)
144
145 rationalQ[e_] := MatchQ[e, _Integer | _Rational];
146 gaussianQ[e_] := rationalQ[e] || (Head[e] === Complex && rationalQ[Re[e]] && rationalQ[Im[e]]);
147 opaqueQ[e_] := MatchQ[e, _Root | _AlgebraicNumber];
148
149 RadicalExpressionQ[e_] := Which[
150   gaussianQ[e], True,
151   AtomQ[e], False,
152   MemberQ[{Plus, Times}, Head[e]], AllTrue[List @@ e, RadicalExpressionQ],
153   Head[e] === Power && Length[e] === 2, rationalQ[e[[2]]] && RadicalExpressionQ[e[[1]]],
154   True, False];
155
156 algebraicFormQ[e_] := Which[
157   gaussianQ[e], True,
158   opaqueQ[e], TrueQ[Precision[e] === Infinity] && FreeQ[e, _Real],
159   AtomQ[e], False,
160   MemberQ[{Plus, Times}, Head[e]], AllTrue[List @@ e, algebraicFormQ],
161   Head[e] === Power && Length[e] === 2, rationalQ[e[[2]]] && algebraicFormQ[e[[1]]],
162   True, False];
163
164 (* exactness is decided by the grammar (algebraic by construction) *)
165 ExactAlgebraicQ[e_] := algebraicFormQ[e] && FreeQ[e, Indeterminate | ComplexInfinity |
166   _DirectedInfinity];
166 exactQ[e_] := ExactAlgebraicQ[e];
167
168 RadicalDepth[e_] := Which[
169   AtomQ[e] || opaqueQ[e], 0,
170   MatchQ[e, Power[_ , _Rational]], 1 + RadicalDepth[First[e]],

```

```

171     True, Max[Prepend[RadicalDepth /@ (List @@ e), 0]]];
172
173 radicalNodes[e_] := If[opaqueQ[e], 0, Count[e, Power[_ , _Rational], {0, Infinity}] -
174     Total[Cases[e, r : (_Root | _AlgebraicNumber) :> Count[r, Power[_ , _Rational], {0, Infinity}
175     ], {0, Infinity}]]];
176
177 bitSize[e_] := Total[Cases[e, n_Integer :> 1 + IntegerLength[Abs[n], 2], {0, Infinity}, Heads ->
178     False]] +
179     Total[Cases[e, r_Rational :> 2 + IntegerLength[Abs[Numerator[r]], 2] + IntegerLength[
180     Denominator[r], 2], {0, Infinity}, Heads -> False]];
181
182 RadicalCost[e_] := {Count[e, _Root | _AlgebraicNumber, {0, Infinity}], RadicalDepth[e],
183     radicalNodes[e], LeafCount[e], bitSize[e]};
184
185 cheaperQ[new_, old_] := Order[RadicalCost[new], RadicalCost[old]] === 1;
186
187 pickCheapest[list_List] := First[SortBy[list, RadicalCost]];
188
189 smallQ[e_] := If[LeafCount[e] > $cfg["MaxLeafCount"], limitHit["MaxLeafCount"]; False, True];
190
191 (* ----- *)
192 (* exact certification *)
193 (* ----- *)
194
195 canonicalNonzeroQ[e_] := e != 0 && MatchQ[e, _Integer | _Rational | _Complex | _Root |
196     _AlgebraicNumber] &&
197     TrueQ[Precision[e] === Infinity];
198
199 (* heuristic pre-rejection by significance arithmetic; never accepts *)
200 numericallyDifferentQ[d_] := Module[{num},
201     If[! TrueQ[$cfg["NumericPrefilter"]], Return[False]];
202     num = bounded[N[d, 40]];
203     TrueQ[NumberQ[num] && Accuracy[num] >= 25 && Abs[num] > 10^-20]];
204
205 certify[a_, b_] := Module[{d, r},
206     bump["Certificates"];
207     If[! exactQ[a] || ! exactQ[b], bump["CertificatesUnknown"]; Return["Unknown"]];
208     If[a == b, bump["CertificatesEqual"]; Return["Equal"]];
209     d = a - b;
210     If[numericallyDifferentQ[d], bump["NumericRejections"]; bump["CertificatesDifferent"];
211     Return["Different"]];
212     r = bounded[RootReduce[d], "Certificate"];
213     Which[
214         r == 0, bump["CertificatesEqual"]; Return["Equal"],
215         canonicalNonzeroQ[r], bump["CertificatesDifferent"]; Return["Different"];
216         r = bounded[PossibleZeroQ[d, Method -> "ExactAlgebraics"], "Certificate"];
217         Which[
218             r == True, bump["CertificatesEqual"]; "Equal",
219             r == False, bump["CertificatesDifferent"]; "Different",
220             True, bump["CertificatesUnknown"]; "Unknown"]];
221
222 SetAttributes[standalone, HoldAll];
223 standalone[body_, failure_] := If[TrueQ[$active], body,
224     Block[{$active = True, $cfg = Association[Options[Strad]], $deadline = AbsoluteTime[] + 20,
225         $stats = newStats[], $limits = <|>, $trace = {}, $records = {}, $memo = <|>, $recursion
226         = 0, $assumptions = True},
227         Quiet[Check[TimeConstrained[MemoryConstrained[body, 1073741824, failure], 20, failure],
228             failure]]];
229
230 EqualityStatus[a_, b_] := standalone[certify[a, b], "Unknown"];
231 CertifiedEqualQ[a_, b_] := EqualityStatus[a, b] === "Equal";
232
233 (* The single acceptance gate. A candidate replaces the incumbent only if it is
234     an explicit radical expression, small, strictly cheaper than the incumbent,
235     and certified equal to the ORIGINAL target of this island. *)
236 accept[candidate_, target_, incumbent_, method_String] := Module[{},
237     If[candidate == $Failed || candidate == target || ! RadicalExpressionQ[candidate] ||
238     ! smallQ[candidate] || ! cheaperQ[candidate, incumbent], Return[incumbent]];
239     If[certify[candidate, target] === "Equal",
240         bump["CandidatesAccepted"];
241         If[Length[$records] < $cfg["MaxTraceEntries"],
242             AppendTo[$records, <|"Before" -> target, "After" -> candidate, "Method" -> method|>]];
243         log[method, ": ", target, " -> ", candidate];
244         trace["Accepted", <|"Method" -> method, "Cost" -> RadicalCost[candidate]|>];
245         candidate,

```

```

236     incumbent]];
237
238 (* variants of a candidate that may print more cheaply; each is re-gated *)
239 acceptWithPolish[candidate_, target_, incumbent_, method_String] := Module[{best = incumbent, v
    },
240     If[candidate === $Failed || ! exactQ[candidate], Return[best]];
241     best = accept[candidate, target, best, method];
242     v = bounded[Simplify[candidate, Assumptions -> True, TimeConstraint -> 5]];
243     best = accept[v, target, best, method <> "/Simplify"];
244     v = bounded[Expand[candidate]];
245     best = accept[v, target, best, method <> "/Expand"];
246     v = bounded[Together[candidate]];
247     best = accept[v, target, best, method <> "/Together"];
248     v = rationalizeRaw[candidate];
249     best = accept[v, target, best, method <> "/Rationalize"];
250     best];
251
252 (* ----- *)
253 (* certified denominator inverse *)
254 (* ----- *)
255
256 validPolynomialQ[p_, x_, minimum_Integer: 1] := Module[{degree},
257     If[! FreeQ[p, $Failed | $Aborted | operationFailed | operationTimedOut] || ! TrueQ[
        PolynomialQ[p, x]], Return[False]];
258     degree = Exponent[p, x];
259     IntegerQ[degree] && degree >= minimum && degree <= $cfg["MaxDegree"]];
260
261 rationalizeRaw[e_] := Module[{t, num, den, p, c, inv, result},
262     t = bounded[Together[e]];
263     If[t === $Failed, Return[$Failed]];
264     num = Numerator[t]; den = Denominator[t];
265     If[rationalQ[den], Return[t]];
266     If[! exactQ[den] || ! smallQ[den], Return[$Failed]];
267     p = bounded[MinimalPolynomial[den, $x]];
268     If[! validPolynomialQ[p, $x], Return[$Failed]];
269     c = CoefficientList[p, $x];
270     If[! AllTrue[c, rationalQ] || First[c] === 0, Return[$Failed]];
271     (* p(d) = 0 gives 1/d = -(a1 + a2 d + ... + an d^(n-1))/a0 (Horner form) *)
272     inv = bounded[-Fold[#1 den + #2 &, Last[c], Reverse[Rest[Most[c]]]]/First[c]];
273     If[inv === $Failed || certify[den inv, 1] != "Equal", Return[$Failed]];
274     result = bounded[Expand[num inv]];
275     If[result === $Failed || ! smallQ[result], $Failed, result]];
276
277 RationalizeDenominator[e_] := standalone[Replace[rationalizeRaw[e], $Failed -> e], e];
278
279 Factorc[e_] := standalone[If[exactQ[e], accept[bounded[Factor[e]], e, e, "Factor"], e], e];
280
281 (* ----- *)
282 (* exact low-degree fast paths *)
283 (* ----- *)
284
285 (* a + b Sqrt[c] with rational a, b, c > 0 nonsquare, after expansion *)
286 quadraticParts[rho_] := Module[{e, terms, rat, irr, a, t, c, b},
287     e = bounded[Expand[rho]];
288     If[e === $Failed, Return[$Failed]];
289     terms = If[Head[e] === Plus, List @@ e, {e}];
290     rat = Select[terms, rationalQ]; irr = Select[terms, ! rationalQ[#] &];
291     If[Length[irr] != 1, Return[$Failed]];
292     a = Total[rat]; t = First[irr];
293     c = Replace[t, {Sqrt[cc_?rationalQ] :> cc, Times[k_?rationalQ, Sqrt[cc_?rationalQ]] :> k^2
        cc, _ -> $Failed}];
294     If[c === $Failed || ! TrueQ[c > 0] || rationalQ[Sqrt[c]], Return[$Failed]];
295     b = Replace[t, {Sqrt[_] :> 1, Times[k_?rationalQ, Sqrt[_] :> Sign[k], _ -> 1}];
296     (* represent as a + b Sqrt[c] with b = +/-1 and c absorbing the coefficient *)
297     {a, b, c}];
298
299 (* direct and indirect denesting of Sqrt[a + b Sqrt[c]] > 0 *)
300 quadraticSquareRoots[{a_, b_, c_}] := Module[{d, s, u, v, e, out = {}},
301     d = a^2 - b^2 c;
302     If[d >= 0,
303         s = Sqrt[d];
304         If[rationalQ[s] && a > 0,

```

```

305     u = (a + s)/2; v = (a - s)/2;
306     If[u >= 0 && v >= 0, AppendTo[out, Sqrt[u] + Sign[b] Sqrt[v]]];
307     (* indirect criterion: -c d must be a rational square *)
308     If[d < 0 && b > 0,
309       e = Sqrt[b^2 - a^2/c];
310       If[rationalQ[e] && 0 <= e <= b,
311         AppendTo[out, c^(1/4) (Sqrt[(b + e)/2] + Sign[a] Sqrt[(b - e)/2])]];
312     out];
313
314     (* real cube roots inside Q(Sqrt[c]): trace-norm criterion *)
315     cubicInQuadratic[{a_, b_, c_}] := Module[{norm, n, roots, out = {}, den, beta},
316       norm = a^2 - b^2 c;
317       n = Sign[norm] Abs[norm]^(1/3);
318       If[! rationalQ[n], Return[{}]];
319       roots = rationalRoots[$x^3 - 3 n $x - 2 a];
320       Do[den = t^2 - n;
321         If[den != 0, beta = t/2 + b Sqrt[c]/den;
322           If[TrueQ[bounded[RootReduce[beta^3 - (a + b Sqrt[c])]] == 0, "Certificate"]], AppendTo[out,
323             beta]],
324       {t, roots}];
325     out];
326     rationalRoots[p_] := Module[{f1},
327       f1 = bounded[FactorList[p]];
328       If[f1 == $Failed || ! ListQ[f1], Return[{}]];
329       DeleteDuplicates[Select[Cases[f1, {f_, _Integer} /; PolynomialQ[f, $x] && Exponent[f, $x]
330         == 1 :>
331         -Coefficient[f, $x, 0]/Coefficient[f, $x, 1]], rationalQ]];
332
333     (* Honsbeek: Sqrt[A + B] with A^3, B^3 nonzero rationals, A + B > 0 real *)
334     honsbeekSquareRoots[rho_] := Module[{terms, a, b, ratio, roots, den, num, out = {}},
335       If[Head[rho] != Plus || Length[rho] != 2, Return[{}]];
336       terms = List @@ rho;
337       If[! AllTrue[terms, MatchQ[#, Power[_?rationalQ, Rational[1, 3]] | Times[_?rationalQ, Power[
338         _?rationalQ, Rational[1, 3]]] &], Return[{}]];
339       {a, b} = Replace[bounded[RootReduce[#^3], "Certificate"], $Failed -> Null] & /@ terms;
340       If[! AllTrue[{a, b}, rationalQ] || a == 0 || b == 0, Return[{}]];
341       ratio = b/a;
342       roots = rationalRoots[$x^4 + 4 $x^3 + 8 ratio $x - 4 ratio];
343       Do[den = b - s^3 a;
344         If[den > 0,
345           num = -s^2 terms[[1]]^2/2 + s terms[[1]] terms[[2]] + terms[[2]]^2;
346           out = Join[out, {num/Sqrt[den], -num/Sqrt[den]}],
347         {s, roots}];
348     out];
349
350     (* Sqrt of a rational combination of several square roots of rationals:
351       solve (Sum x_u Sqrt[u])^2 == rho over the rationals *)
352     multiSurdSquareRoots[rho_] := Module[{e, surds, primes, unknownSets, out = {}, vars, cand, sq,
353       basis, eqs, sol, ratpart},
354       e = bounded[Expand[rho]];
355       If[e == $Failed, Return[{}]];
356       surds = DeleteDuplicates[Cases[e, Power[_Integer, Rational[1, 2]] /; n > 1 :> n, {0,
357         Infinity}]];
358       If[Length[surds] < 2 || Length[surds] > 15, Return[{}]];
359       If[! AllTrue[If[Head[e] == Plus, List @@ e, {e}],
360         MatchQ[#, _?rationalQ | Power[_Integer, Rational[1, 2]] | Times[_?rationalQ, Power[
361         _Integer, Rational[1, 2]]] &], Return[{}]];
362       primes = Union @@ (First /@ FactorInteger[#] & /@ surds);
363       unknownSets = DeleteDuplicates[{Union[{1}, primes], Union[{1}, primes, surds]}];
364       ratpart[ex_] := ex /. Power[_Integer, Rational[1, 2]] -> 0;
365       Do[
366         If[Length[U] > 16, Continue[]];
367         vars = Array[cvar, Length[U]];
368         cand = vars . Sqrt[U];
369         sq = Expand[cand^2];
370         basis = Union[surds, DeleteDuplicates[Cases[sq, Power[_Integer, Rational[1, 2]] :> n, {0,
371           Infinity}]]];
372         eqs = Prepend[Table[Coefficient[sq, Sqrt[bb]] == Coefficient[e, Sqrt[bb]], {bb, basis}],
373           ratpart[sq] == ratpart[e]];
374         sol = bounded[Solve[eqs, vars, Rationals]];
375         If[ListQ[sol] && sol != {},

```

```

369     out = Join[out, DeleteDuplicates[cand /. sol]];
370     Break[]];
371     {U, unknownSets}}];
372     out];
373
374 (* Gaussian square root: Sqrt[a + b I] with a^2 + b^2 a rational square *)
375 gaussianSquareRoots[z_] := Module[{a = Re[z], b = Im[z], n},
376   n = Sqrt[a^2 + b^2];
377   If[rationalQ[n], {Sqrt[(n + a)/2] + I Sign[b] Sqrt[(n - a)/2]}, {}]];
378
379 (* roots of unity as rational powers of -1 *)
380 unity[k_Integer, q_Integer] := (-1)^(Mod[2 k, 2 q]/q);
381
382 (* candidates for rho^(1/q), rho exact, produced by the shape-specific recipes *)
383 rootCandidates[rho_, q_Integer] := Module[{parts, out = {}, sign = 1, r = rho, neg},
384   If[q == 2 && Head[rho] == Complex, Return[gaussianSquareRoots[rho]]];
385   neg = TrueQ[bounded[rho < 0, "Certificate"]];
386   If[neg, r = -rho];
387   Which[
388     q == 2,
389     parts = quadraticParts[r];
390     If[parts != $Failed, out = quadraticSquareRoots[parts]];
391     out = Join[out, honsbeekSquareRoots[r], multiSurdSquareRoots[r]];
392     If[neg, out = I out],
393     q == 3,
394     parts = quadraticParts[r];
395     If[parts != $Failed, out = cubicInQuadratic[parts]];
396     If[neg, out = (-1)^(1/3) out]];
397   out];
398
399 (* ----- *)
400 (* Kummer-linear factors and index reduction *)
401 (* ----- *)
402
403 (* the radicals (and I) occurring in an expression, as an explicit Extension
404    specification: factoring over them keeps roots in radical presentation,
405    whereas Extension -> Automatic may canonicalize them into Root objects *)
406 radicalExtension[e_] := Module[{ext = DeleteDuplicates[Cases[e, Power[_ , _Rational], {0,
407   Infinity}]]},
408   If[! FreeQ[e, Complex], ext = Prepend[ext, I]];
409   If[ext == {}, Automatic, ext]];
410
411 (* radical presentations of an algebraic number given with Root objects *)
412 radicalForms[z_] := Module[{out = {z}, r},
413   If[FreeQ[z, _Root | _AlgebraicNumber], Return[out]];
414   r = bounded[ToRadicals[z]];
415   If[r != $Failed, AppendTo[out, r]];
416   r = bounded[RootReduce[z], "Certificate"];
417   If[r != $Failed, r = bounded[ToRadicals[r]]; If[r != $Failed, AppendTo[out, r]]];
418   DeleteDuplicates[Select[out, exactQ]];
419
420 (* roots gamma of x^k == rho lying in the field generated by the radicals of rho *)
421 linearRoots[rho_, k_Integer] := Module[{f1, roots},
422   f1 = bounded[FactorList[x^k - rho, Extension -> radicalExtension[rho]]];
423   If[f1 == $Failed || ! ListQ[f1],
424     f1 = bounded[FactorList[x^k - rho, Extension -> Automatic]];
425   If[f1 == $Failed || ! ListQ[f1], Return[{}]];
426   roots = Cases[f1, {f_, _Integer} /; PolynomialQ[f, $x] && Exponent[f, $x] == 1 :>
427     Together[-Coefficient[f, $x, 0]/Coefficient[f, $x, 1]]];
428   DeleteDuplicates[Select[Flatten[radicalForms /@ roots], exactQ]];
429
430 (* denest a sub-problem recursively under the shared budget *)
431 recurse[sub_] := If[$recursion >= $cfg["MaxRecursion"] || expiredQ[], sub,
432   Block[{$recursion = $recursion + 1, $cfg = Append[$cfg, "MaxTrials" -> Quotient[$cfg["
433     MaxTrials"], 4]}],
434     improveNumber[sub]]];
435
436 (* candidates for target == rho^(p/q) from linear factors of x^k - rho, k | q *)
437 kummerCandidates[target_, rho_, p_Integer, q_Integer] := Module[{best = target, gammas, k, rest,
438   sub, den},
439   Do[
440     If[expiredQ[], Break[]];

```

```

438   gammas = linearRoots[rho, k];
439   rest = q/k;
440   Do[
441     If[expiredQ[], Break[]];
442     If[rest == 1,
443       Do[best = acceptWithPolish[bounded[Expand[(gamma unity[1, q])^p]], target, best, "
LinearFactor"], {1, 0, q - 1}],
444       (* rho = gamma^k, so target = (gamma zeta_k^j)^(p/rest) zeta_rest^l for some j, l *)
445       Do[
446         den = bounded[Expand[gamma unity[j, k]]];
447         If[den == $Failed, Continue[]];
448         sub = recurse[den^(1/rest)];
449         If[sub != den^(1/rest) && RadicalDepth[sub] < RadicalDepth[target],
450           Do[best = acceptWithPolish[bounded[Expand[(sub unity[1, rest])^p]], target, best, "
IndexReduction"], {1, 0, rest - 1}],
451           {j, 0, k - 1}],
452         {gamma, gammas}];
453     If[RadicalDepth[best] < RadicalDepth[target], Break[]],
454     {k, Reverse[Rest[Divisors[q]]]};
455   best];
456
457   (* negative real radicand: rho^(p/q) = (-1)^(p/q) (-rho)^(p/q) *)
458   negativeRadicandCandidate[target_, rho_, p_Integer, q_Integer] := Module[{sub},
459     If[! TrueQ[bounded[rho < 0, "Certificate"]], Return[target]];
460     sub = recurse[(-rho)^(p/q)];
461     If[sub == (-rho)^(p/q), target, accept[bounded[Expand[(-1)^(p/q) sub]], target, target, "
NegativeRadicand"]];
462
463   (* ----- *)
464   (* the multiplier search (bounded FIFO queue, single admission gate) *)
465   (* ----- *)
466
467   reductionIndex[e_] := Module[{d = RadicalDepth[e], nodes, q = 1},
468     nodes = Select[Cases[e, Power[_ , _Rational], {0, Infinity}], RadicalDepth[#] == d &];
469     Do[q = LCM[q, Denominator[Last[node]]],
470       If[q > $cfg["MaxRootIndex"], limitHit["MaxRootIndex"]; Return[$Failed]], {node, nodes}];
471     If[q < 2, $Failed, q];
472
473   complementaryFactor[Power[b_, r_Rational]] := b^(Mod[-Numerator[r], Denominator[r]]/Denominator
[r]);
474   complementaryFactor[Complex[0, _]] := -I;
475   complementaryFactor[e_] := Module[{deep = Cases[e, Power[_ , _Rational], {0, Infinity}], maxd},
476     If[deep == {}, 1, maxd = Max[RadicalDepth /@ deep];
477     Times @@ (complementaryFactor /@ Select[deep, RadicalDepth[#] == maxd &])];
478   complementaryMultiplier[term_] := Times @@ (complementaryFactor /@ If[Head[term] == Times,
List @@ term, {term}]);
479
480   multiplierSearch[target_, initialBest_] := Module[
481     {best = initialBest, q, rho, queue = {}, seen = <||>, admitted = 0, proposed = 0, cursor = 1,
482
483     cap = $cfg["MultiplierCap"], proposalCap, admit, seeds, terms, m, theta, p, degree, gcd, gd,
484     roots,
485     mroot, candidate, disc, primes, j, digits, batch, sol, trials = 0, sinceImprovement = 0,
486     bestDegree = Infinity},
487     If[cap == 0 || $cfg["MaxTrials"] == 0 || expiredQ[], Return[best]];
488     q = reductionIndex[target]; If[q == $Failed, Return[best]];
489     rho = bounded[Expand[target^q]]; If[rho == $Failed || ! exactQ[rho], Return[best]];
490     proposalCap = 4 cap;
491     (* the only insertion point: caps admissions, counts proposals, dedups by exact canonical
value *)
492     admit[value_] := Module[{canonical, key},
493       If[admitted >= cap, limitHit["MultiplierCap"]; Return[False]];
494       If[proposed >= proposalCap, limitHit["MultiplierProposals"]; Return[False]];
495       proposed++; bump["MultipliersProposed"];
496       If[! exactQ[value] || ! smallQ[value], Return[False]];
497       (* the key is the expanded form: algebraically equal multipliers written
differently are rare among products of primes and visible radicals, and a
RootReduce per proposal would dominate the cost of a trial *)
498       canonical = bounded[Expand[value]];
499       If[canonical == $Failed || TrueQ[canonical == 0], Return[False]];
500       key = ToString[canonical, InputForm];
501       If[KeyExistsQ[seen, key], bump["DuplicateMultipliers"]; Return[False]];

```

```

501 AssociateTo[seen, key -> True]; AppendTo[queue, value];
502 admitted++; bump["MultipliersAdmitted"]; True];
503 If[ListQ[$cfg["Multipliers"]],
504 seeds = $cfg["Multipliers"],
505 terms = DeleteCases[Replace[#, {Times[a____, _?rationalQ, b____] :> a*b, _?rationalQ -> 0,
Complex[a_, b_] :> Sign[b] I}] & /@
506 If[Head[rho] === Plus, List @@ rho, {rho}], 0];
507 seeds = Join[{1}, DeleteCases[complementaryMultiplier /@ terms, 1], {2, 3, 5}];
508 Do[If[admitted >= cap || proposed >= proposalCap || expiredQ[], Break[]]; admit[seed], {seed,
seeds}];
509 While[cursor <= Length[queue] && ! expiredQ[],
510 (* "MaxTrials" bounds the trials spent on this island; "Patience" stops the
511 search when an improvement has been found and further trials do not help *)
512 If[trials >= $cfg["MaxTrials"], limitHit["MaxTrials"]; Break[]];
513 If[best != initialBest && sinceImprovement >= $cfg["Patience"], limitHit["Patience"];
Break[]];
514 m = queue[cursor]; cursor++; trials++; sinceImprovement++; bump["Trials"];
515 (* the multiplied radicand keeps its radical presentation: the GCD roots then
516 come out in the radicals of rho, not as opaque algebraic numbers *)
517 theta = bounded[Expand[m rho]];
518 If[theta === $Failed, Continue[]];
519 p = bounded[MinimalPolynomial[theta^(1/q), $x]];
520 If[! validPolynomialQ[p, $x], If[p != $Failed, limitHit["MaxDegree"]; Continue[]];
521 degree = Exponent[p, $x];
522 log["multiplier ", m, " minpoly degree ", degree];
523 bestDegree = Min[bestDegree, degree];
524 trace["Trial", <|"Multiplier" -> m, "Degree" -> degree, "Index" -> q|>];
525 (* roots of x^q = m rho: linear factors over the radicals of the radicand, then the
526 proper GCD factor with the minimal polynomial of the principal root *)
527 roots = If[m === 1, {}, linearRoots[theta, q]];
528 gcd = bounded[PolynomialGCD[p, $x^q - theta, Extension -> Automatic]];
529 If[validPolynomialQ[gcd, $x],
530 gd = Exponent[gcd, $x];
531 If[gd < q,
532 roots = Join[roots, Which[
533 gd === 1, {-Coefficient[gcd, $x, 0]/Coefficient[gcd, $x, 1]},
534 gd <= $cfg["MaxSolveDegree"],
535 sol = bounded[Solve[gcd == 0, $x]];
536 If[ListQ[sol], Select[$x /. sol, exactQ], {}],
537 True, {}]]];
538 roots = DeleteDuplicates[Select[Flatten[radicalForms /@ roots], exactQ]];
539 If[roots != {},
540 Module[{},
541 mroot = bounded[m^(1/q)];
542 If[mroot != $Failed,
543 Do[If[expiredQ[], Break[]];
544 Do[If[expiredQ[], Break[]];
545 candidate = bounded[Expand[z/mroot unity[k, q]]];
546 Module[{before = best},
547 best = acceptWithPolish[candidate, target, best, "MultiplierOrbit"];
548 If[best != before, sinceImprovement = 0];
549 (* a certified but not cheaper candidate (e.g. gamma^(1/3) with gamma in Q(rho))
550 may itself be denestable: try it recursively *)
551 If[candidate != $Failed && RadicalDepth[candidate] <= RadicalDepth[target] &&
candidate != target &&
552 certify[candidate, target] === "Equal",
553 Module[{before = best},
554 best = acceptWithPolish[recurve[candidate], target, best, "MultiplierOrbit/Recursive
555 "];
556 If[best != before, sinceImprovement = 0]],
557 {k, 0, q - 1}],
558 {z, roots}]]];
559 If[RadicalDepth[best] <= 1, Break[]];
560 (* discriminant primes of a promising trial (smallest degree seen so far): stream
561 prime powers and a bounded prefix of mixed-radix products *)
562 If[! ListQ[$cfg["Multipliers"]] && degree <= bestDegree && admitted < cap && proposed <
proposalCap && ! expiredQ[],
563 disc = bounded[Discriminant[p, $x]];
564 If[IntegerQ[disc] && disc != 0,
565 primes = Take[Select[First /@ FactorInteger[Abs[disc]], PrimeQ], UpTo[8]];
566 batch = 0;
567 Do[If[admitted >= cap || proposed >= proposalCap || batch >= 24 || expiredQ[], Break[]];

```

```

567     Do[batch++; admit[m prime^e], {e, 1, q - 1}], {prime, primes}];
568     j = 1;
569     While[primes != {} && j < q^Length[primes] && batch < 24 && admitted < cap && proposed <
proposalCap && ! expiredQ[],
570       digits = IntegerDigits[j, q, Length[primes]]; j++; batch++;
571       admit[m Times @@ MapThread[Power, {primes, digits}]]];
572     If[admitted >= cap, limitHit["MultiplierCap"]];
573     If[trials >= $cfg["MaxTrials"], limitHit["MaxTrials"]];
574     best];
575
576     (* ----- *)
577     (* one exact algebraic island *)
578     (* ----- *)
579
580     (* whole-island proposals that do not depend on the shape *)
581     genericProposals[target_, incumbent_] := Module[{best = incumbent, r, mp, solver},
582       solver = $cfg["Solver"];
583       If[solver != Automatic,
584         r = bounded[Catch[solver[target], _, $Failed &]];
585         best = acceptWithPolish[r, target, best, "Solver"];
586       If[! FreeQ[target, _Root | _AlgebraicNumber],
587         r = bounded[ToRadicals[target]];
588         If[r != $Failed && r != target, best = acceptWithPolish[recurse[r], target, best, "
ToRadicals"]];
589       r = bounded[RootReduce[target], "Certificate"];
590       If[r != $Failed,
591         best = accept[r, target, best, "RootReduce"];
592         If[opaqueQ[r] && Head[r] === Root && PolynomialQ[First[r][$x], $x] && Exponent[First[r][$x],
$х] <= 4,
593           best = acceptWithPolish[bounded[ToRadicals[r]], target, best, "RootReduce/ToRadicals"]];
594       best = accept[bounded[Simplify[target, Assumptions -> True, TimeConstraint -> 5]], target,
best, "Simplify"];
595       If[TrueQ[$cfg["Factor"]], best = accept[bounded[Factor[target]], target, best, "Factor"]];
596       best = accept[rationalizeRaw[target], target, best, "Rationalize"];
597       best];
598
599     (* fast paths for a single rational-power node *)
600     powerProposals[target_, incumbent_] := Module[{best = incumbent, rho, p, q, root, cand},
601       If[! MatchQ[target, Power[_ , _Rational]], Return[best]];
602       rho = First[target]; p = Numerator[Last[target]]; q = Denominator[Last[target]];
603       If[q > $cfg["MaxRootIndex"], limitHit["MaxRootIndex"]; Return[best]];
604       (* shape-specific recipes for the q-th root, then the integer power p *)
605       cand = rootCandidates[rho, q];
606       Do[best = acceptWithPolish[bounded[Expand[c^p]], target, best, "FastPath"], {c, cand}];
607       If[RadicalDepth[best] <= 1, bump["FastPathAccepted"]; Return[best]];
608       (* negative real radicand: separate the phase, denest the modulus *)
609       best = negativeRadicandCandidate[target, rho, p, q];
610       If[RadicalDepth[best] <= 1, Return[best]];
611       (* roots of rho inside Q(rho), and index reduction through divisors of q *)
612       best = kummerCandidates[target, rho, p, q];
613       (* an even index with a quadratic recipe available for the square root *)
614       If[RadicalDepth[best] >= 2 && EvenQ[q] && q > 2,
615         root = recurse[rho^(1/2)];
616         If[root != rho^(1/2) && RadicalDepth[root] < RadicalDepth[target],
617           root = recurse[root^(2/q)];
618         best = acceptWithPolish[bounded[Expand[root^p]], target, best, "EvenIndexSplit"]];
619       best];
620
621     (* the multiplier search is meant for a radical node or a product of radical
622       nodes; a sum island gets the whole-island proposals only *)
623     searchableQ[e_] := MatchQ[e, Power[_ , _Rational]] ||
624       (Head[e] === Times && AllTrue[List @@ e, MatchQ[#, _?gaussianQ | Power[_ , _Rational]] &]);
625
626     (* results for one island are memoized within the session: the same sub-problem
627       recurs through roots of unity, index reduction and repeated passes *)
628     improveNumber[target_] := Module[{best = target, key},
629       If[expiredQ[] || ! exactQ[target] || ! smallQ[target], Return[target]];
630       If[RadicalDepth[target] < 2 && FreeQ[target, _Root | _AlgebraicNumber], Return[target]];
631       key = target;
632       If[KeyExistsQ[$memo, key], Return[$memo[key]]];
633       bump["Islands"];
634       best = powerProposals[target, best];

```

```

635   If[RadicalDepth[best] >= 2 || ! FreeQ[best, _Root | _AlgebraicNumber], best =
        genericProposals[target, best]];
636   If[RadicalDepth[best] >= 2 && $cfg["Solver"] === Automatic && searchableQ[target] && !
        expiredQ[],
637     best = multiplierSearch[target, best]];
638   If[! expiredQ[], AssociateTo[$memo, key -> best]];
639   best];
640
641   (* an exact algebraic island: children first (all levels) or outermost
642      radical nodes first (default), then the island as a whole *)
643   (* recombination of an island whose components were rewritten: the proposals
644      are computed from the rewritten form but certified against the original *)
645   combineProposals[target_, incumbent_] := Module[{best = incumbent, r},
646     r = bounded[RootReduce[incumbent], "Certificate"];
647     best = accept[r, target, best, "Combine/RootReduce"];
648     best = accept[bounded[Simplify[incumbent, Assumptions -> True, TimeConstraint -> 5]], target,
        best, "Combine/Simplify"];
649     best = accept[bounded[Together[incumbent]], target, best, "Combine/Together"];
650     best = accept[bounded[Expand[incumbent]], target, best, "Combine/Expand"];
651     best];
652
653   island[e_] := Module[{current = e, rebuilt},
654     If[expiredQ[], Return[e]];
655     If[RadicalDepth[e] < 2 && FreeQ[e, _Root | _AlgebraicNumber], Return[e]];
656     If[MatchQ[e, Power[_], _Rational],
657       If[TrueQ[$cfg["AllLevels"]],
658         (* the radicand is an island of its own; congruence carries its certificate *)
659         rebuilt = Power[island[First[e]], Last[e]];
660         If[rebuilt != e && cheaperQ[rebuilt, e], current = rebuilt]];
661       Return[improveNumber[current]]];
662     If[MemberQ[{Plus, Times}, Head[e]] || MatchQ[e, Power[_], _Integer],
663       (* components first: each component certified against itself *)
664       rebuilt = If[Head[e] === Power, Power[island[First[e]], Last[e]], Map[island, e]];
665       If[rebuilt != e && cheaperQ[rebuilt, e], current = rebuilt];
666       If[RadicalDepth[current] >= 2 || ! FreeQ[current, _Root | _AlgebraicNumber],
667         rebuilt = improveNumber[current];
668         If[rebuilt != current, current = rebuilt]];
669       If[current != e, current = combineProposals[e, current]];
670       Return[current]];
671     If[opaqueQ[e], Return[improveNumber[e]]];
672     e];
673
674   (* ----- *)
675   (* host traversal by congruence *)
676   (* ----- *)
677
678   walk[e_] := Module[{h},
679     If[expiredQ[], Return[e]];
680     If[exactQ[e], Return[island[e]]];
681     If[AtomQ[e], Return[e]];
682     h = Head[e];
683     Which[
684       MemberQ[{List, Plus, Times}, h], Map[walk, e],
685       h === Power && Length[e] === 2 && MatchQ[Last[e], _Integer | _Rational], Power[walk[First[e]], Last[e]],
686       True, e];
687
688   (* ----- *)
689   (* the session *)
690   (* ----- *)
691
692   run[e_, cfg_Association, coreOnly_> False] :=
693     Block[{$active = True, $cfg = cfg, $deadline = AbsoluteTime[] + cfg["TimeBudget"],
694       $stats = newStats[], $limits = <|>, $trace = {}, $records = {}, $memo = <|>, $recursion =
695         0, $assumptions = True},
696       Module[{snapshot = e, candidate, outcome = Null, started = AbsoluteTime[], passes, status,
697         committed},
698         passes = If[TrueQ[cfg["AllLevels"]] && ! coreOnly, cfg["MaxPasses"], 1];
699         Which[
700           TrueQ[cfg["TimeBudget"] == 0], limitHit["TimeBudget"],
701           ! FreeQ[e, _Real | _Complex?InexactNumberQ], limitHit["InexactInput"],
702           coreOnly && ! exactQ[e], limitHit["NotExactAlgebraic"],

```

```

701     True,
702     outcome = TimeConstrained[
703       MemoryConstrained[
704         Do[
705           If[expiredQ[], Break[]];
706           candidate = If[coreOnly, improveNumber[snapshot], walk[snapshot]];
707           bump["PassesCompleted"];
708           If[candidate == snapshot || ! cheaperQ[candidate, snapshot], Break[]];
709           (* whole-input certificate when the whole input is a number; hosts rely on congruence
710             *)
711           If[exactQ[e] && certify[candidate, e] != "Equal", Break[]];
712           snapshot = candidate;
713           If[pass == passes && passes > 1, limitHit["MaxPasses"]],
714           {pass, passes}],
715           cfg["MemoryBudget"], memoryStopped],
716           cfg["TimeBudget"], timeStopped];
717       If[outcome == memoryStopped, limitHit["MemoryBudget"]];
718       If[outcome == timeStopped, limitHit["TimeBudget"]];
719     status = Which[
720       outcome == timeStopped, "Timeout",
721       outcome == memoryStopped, "MemoryLimit",
722       TrueQ[cfg["TimeBudget"] == 0], "Disabled",
723       snapshot == e, "Unchanged",
724       True, "Improved"];
725     <|"Result" -> snapshot, "Status" -> status, "Limits" -> Keys[$limits],
726     "InitialCost" -> RadicalCost[e], "FinalCost" -> RadicalCost[snapshot],
727     "Statistics" -> $stats, "Certificates" -> $records,
728     "ElapsedSeconds" -> AbsoluteTime[] - started, "Options" -> cfg,
729     "Trace" -> $trace, "CompletenessClaim" -> False|>]];
730   invoke[e_, head_Symbol, rules_List, report_, core_] := Module[{cfg, result},
731     cfg = resolveOptions[head, rules];
732     If[FailureQ[cfg], Return[cfg]];
733     result = run[e, cfg, core];
734     If[TrueQ[report], result, result["Result"]]];
735
736   Strad[e_, all : (True | False), opts : OptionsPattern[]] := invoke[e, Strad, {"AllLevels" ->
737     all, opts}, False, False];
738   Strad[e_, opts : OptionsPattern[]] := invoke[e, Strad, {opts}, False, False];
739   DenestRadicals[e_, all : (True | False), opts : OptionsPattern[]] := invoke[e, DenestRadicals, {
740     "AllLevels" -> all, opts}, False, False];
741   DenestRadicals[e_, opts : OptionsPattern[]] := invoke[e, DenestRadicals, {opts}, False, False];
742   DenestCore[e_, opts : OptionsPattern[]] := invoke[e, DenestCore, {opts}, False, True];
743   DenestReport[e_, all : (True | False), opts : OptionsPattern[]] := invoke[e, DenestReport, {
744     "AllLevels" -> all, opts}, True, False];
745   DenestReport[e_, opts : OptionsPattern[]] := invoke[e, DenestReport, {opts}, True, False];
746
747   End[];
748   EndPackage[];

```

B Files of this directory

File	Purpose
unified_analysis_B.tex, .pdf	This document and its source; the appendix reads <code>../../corrected/StradFixed2.wl</code> .
tables/*.tex	Result tables generated by <code>harness/export_tables2.wl</code> .
harness/runner2.wl	Runner for the battery (loads the package, captures values, messages, times, depths).
harness/exp_battery2.wl	The unified-A battery (its <code>exp_fixed.wl</code>) on the new version.
harness/exp_fuzz2.wl	The ten randomized families, same seed.

File	Purpose
<code>harness/probe_gaps2_1..4.wl</code>	The probing scripts of <code>KNOWN_GAPS.md</code> on the new version.
<code>harness/probe_reviews_fixed.wl</code>	The reviews' probes against <code>StradFixed.wl</code> , executed.
<code>harness/run_review_suite.wl</code>	Runs a review's own suite against its own proposal (argument 4, 5 or 6).
<code>harness/run_all.sh</code>	First sequential chain: battery, families, probes.
<code>harness/run_all2.sh</code>	Second chain: the reviews' suites, the regression suite, the table export.
<code>harness/run_all3.sh</code>	Third chain: the reviews' probes against <code>StradFixed.wl</code> .
<code>tests/StradFixed2.wlt</code>	The regression suite.
<code>tests/run_tests.wls</code>	Its fresh-kernel runner.
<code>logs/*.txt</code>	Raw transcripts of every run reported here.